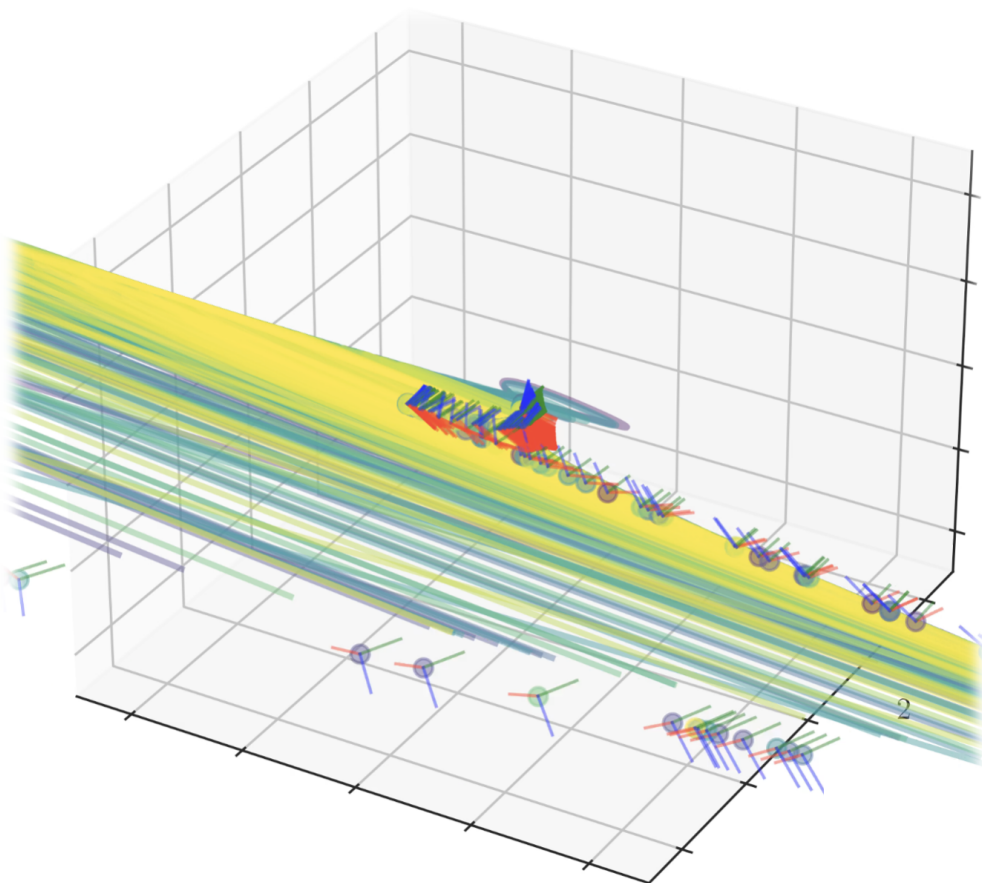

Meta-Learning for Quadcopter Control

Cyril Goffin

Supervised by Daniele Martinelli and Nicolas Kirsch

Prof. G. Ferrari Trecate



January 19th, 2026

Contents

1	Introduction	1
2	Problem Formulation	2
2.1	Quadcopter model	2
2.2	Control problem	5
2.2.1	Disturbance modeling	6
2.2.2	Neural network policy	6
2.2.3	Cost function and optimization problem	7
3	Model-Agnostic Meta-Learning	8
3.1	Meta-learning background and related work	8
3.2	Optimization-based meta-learning and MAML	8
3.3	MAML algorithm	9
3.3.1	Meta-training procedure	10
3.3.2	Few-shot adaptation on a new task	10
3.3.3	Baseline: joint training	10
3.3.4	Task definition for quadcopter control	10
4	Numerical Results	12
4.1	Neural network sanity check	12
4.2	Motivation experiments	13
4.3	MAML experiments with linearized dynamics	16
4.3.1	Wind disturbance: single-axis	16
4.3.2	Wind disturbance: all directions	19
4.3.3	Robustness analysis	22
4.4	MAML experiments with nonlinear dynamics	23
4.4.1	Regulation under nonlinear dynamics	23
4.4.2	Wind disturbance: single-axis	24
4.4.3	Wind disturbance: all directions	26
4.5	Additional observations and implementation notes	28
5	Conclusion	29
	References	30
	Appendix	31
5.1	LQR formulation	31
5.2	Gantt chart	32

1. Introduction

Autonomous quadcopters operate in environments where external disturbances, such as wind, are difficult to model accurately and may vary significantly over time. Designing controllers that are both robust to such disturbances and capable of adapting to new conditions remains a central challenge in control and robotics. While classical control methods rely on accurate models and extensive tuning, learning-based approaches offer greater flexibility but typically require large amounts of data and struggle to generalize across varying disturbance conditions.

Meta-learning provides a principled framework to address this limitation by shifting the objective from learning a single optimal controller to learning an initialization that can be efficiently adapted to new tasks. Rather than requiring extensive retraining for each new disturbance configuration, meta-learning explicitly optimizes for fast adaptation from a small number of samples.

This project investigates whether meta-learning can enable quadcopter controllers to adapt rapidly to new environments using only a small amount of data. In particular, the Model-Agnostic Meta-Learning (MAML) framework is applied to a neural network-based controller implemented as a multi-layer perceptron (MLP) for wind disturbance rejection. All experiments are conducted in simulation, allowing for systematic evaluation under controlled and reproducible conditions. The study proceeds by first validating neural network controllers on basic regulation tasks, then highlighting the limitations of standard joint training when data availability is unbalanced across disturbance conditions. Building on this motivation, meta-learning is applied to increasingly challenging scenarios, including wind disturbances along a single axis and from arbitrary directions. Both linearized and full nonlinear quadcopter dynamics are considered, with the use of the nonlinear model motivated by the goal of reducing the sim-to-real gap and enabling potential future deployment in real-world scenarios.

The project follows a clearly defined timeline, with the corresponding Gantt chart provided in Appendix 5.2. All code, results, and experimental material used in this work are available at <https://github.com/DecodEPFL/MetaLearning-Control-Quadcopters>.

2. Problem Formulation

In this chapter, the quadcopter system and the associated control problem are formally defined. First, a mathematical model of the quadcopter dynamics is introduced, together with the assumptions and simplifications adopted for simulation and control design. The resulting model is then used to formulate the control objective in the presence of external disturbances, leading to a constrained control problem.

2.1 Quadcopter model

A mathematical model of the quadcopter is required in order to simulate its dynamics, design control laws, and evaluate the performance of learning-based approaches such as meta-learning. In the literature, a wide range of quadcopter models can be found, spanning from simple linearized representations around hover to highly nonlinear and state-dependent formulations that capture strong coupling effects and enable aggressive flight maneuvers. For instance, Ahmad *et al.* propose a simplified linear model based on small-angle assumptions and decoupled translational and rotational dynamics, primarily intended for LQR-based control design and simulation studies [1]. In contrast, Kamel *et al.* employ a more realistic nonlinear rigid-body model augmented with aerodynamic drag, wind disturbances, and identified inner-loop dynamics, enabling accurate trajectory tracking using linear and nonlinear MPC on real aerial platforms [2]. An even more advanced modeling approach is presented by Foehn and Scaramuzza, who use a quaternion-based formulation and a state-dependent linearization to design a unified LQR controller capable of handling strongly coupled translational and rotational dynamics during agile flight [3].

For the purpose of the present project, the objective is not to achieve the highest possible physical fidelity in quadcopter modeling, but rather to provide a sufficiently representative dynamical system on which control strategies and meta-learning techniques can be developed and evaluated. Consequently, the simplified quadcopter model introduced by Ahmad *et al.* was selected, as it offers a clear and tractable formulation while remaining adequate for simulation-based control and learning experiments. The dynamics of this simplified quadcopter model are presented below.

Let's first consider the schematic representation of a quadcopter shown in Figure 2.1, where F_1 , F_2 , F_3 , and F_4 are the upward thrust forces respectively generated by the angular speeds ω_1 , ω_2 , ω_3 , and ω_4 . The four arms of the quadcopter are symmetrical and their length is denoted by l . Roll, pitch and yaw angles about the x_B , y_B and z_B axes are denoted by ϕ , θ , and ψ , respectively.

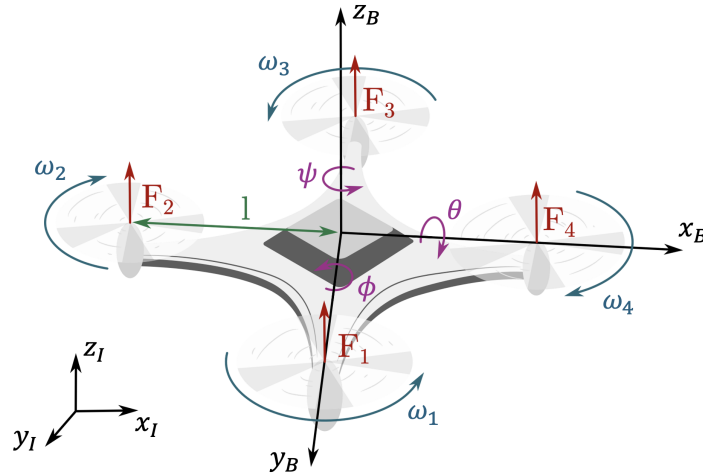


Figure 2.1: Schematic representation of a quadcopter.

In Figure 2.1, the inertial frame (fixed on the ground) is represented by (x_I, y_I, z_I) , while the body frame (fixed on the quadcopter) is represented by (x_B, y_B, z_B) . The rotation matrix between these two frames is expressed as:

$$R = \begin{bmatrix} \cos \theta \cos \psi & \sin \phi \sin \theta \cos \psi - \cos \phi \sin \psi & \cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi \\ \cos \theta \sin \psi & \sin \phi \sin \theta \sin \psi + \cos \phi \cos \psi & \cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi \\ -\sin \theta & \sin \phi \cos \theta & \cos \phi \cos \theta \end{bmatrix}. \quad (2.1)$$

It should be noted that motors 2 and 4 rotate clockwise, while motors 1 and 3 rotate counter-clockwise. This counter-rotating configuration is necessary to balance the reaction torques generated by the propellers and to prevent the vehicle from experiencing an uncommanded yaw motion. By appropriately varying the relative speeds of the clockwise and counter-clockwise rotating motors, a net yaw torque can be produced, allowing controlled rotation about the vertical axis while maintaining overall torque equilibrium.

While the difference between ω_1 and ω_3 provides the torque τ_x about the x -axis, the difference between ω_2 and ω_4 provides the torque τ_y about the y -axis. Yaw motion is obtained when there is a difference in the angular speeds of the two pairs of propellers (one pair rotating clockwise and the other counter-clockwise). Those torques are thus expressed as:

$$\tau_x = k_t l (\omega_1^2 - \omega_3^2), \quad (2.2)$$

$$\tau_y = k_t l (\omega_2^2 - \omega_4^2), \quad (2.3)$$

$$\tau_z = k_d (\omega_2^2 + \omega_4^2 - \omega_1^2 - \omega_3^2), \quad (2.4)$$

where k_t and k_d are the thrust and drag coefficients, respectively, which depend on air density, propeller geometry, and blade radius.

According to Euler's equations, the torques about the x , y , and z axes are represented as:

$$\begin{bmatrix} \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} I_x \ddot{\phi} \\ I_y \ddot{\theta} \\ I_z \ddot{\psi} \end{bmatrix} + \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} \times \begin{bmatrix} I_x \dot{\phi} \\ I_y \dot{\theta} \\ I_z \dot{\psi} \end{bmatrix}, \quad (2.5)$$

where I_x , I_y , and I_z denote the principal moments of inertia of the quadcopter about the body-fixed x , y , and z axes, respectively. The cross-product term captures the gyroscopic coupling between the rotational axes, which arises from the rigid-body dynamics of the quadcopter.

According to Newton's equation of translational motion:

$$m \begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} = R \begin{bmatrix} 0 \\ 0 \\ F \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix}, \quad (2.6)$$

where $F = F_1 + F_2 + F_3 + F_4$ and each rotor thrust $F_i = k_t \omega_i^2$ ($i = 1, 2, 3, 4$). The parameters m and g denote the mass of the quadcopter and the gravitational acceleration, respectively.

Combining (2.1)–(2.6) yields the nonlinear dynamic model of the quadcopter:

$$m \ddot{x} = \underbrace{(F_1 + F_2 + F_3 + F_4)}_F (\cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi), \quad (2.7)$$

$$m \ddot{y} = \underbrace{(F_1 + F_2 + F_3 + F_4)}_F (\cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi), \quad (2.8)$$

$$m \ddot{z} = \underbrace{(F_1 + F_2 + F_3 + F_4)}_F (\cos \phi \cos \theta) - mg, \quad (2.9)$$

$$I_x \ddot{\phi} = \underbrace{l(F_1 - F_3)}_{\tau_x} + (I_y - I_z) \dot{\theta} \dot{\psi}, \quad (2.10)$$

$$I_y \ddot{\theta} = \underbrace{l(F_2 - F_4)}_{\tau_y} + (I_z - I_x) \dot{\phi} \dot{\psi}, \quad (2.11)$$

$$I_z \ddot{\psi} = \underbrace{k_d/k_t (F_2 + F_4 - F_1 - F_3)}_{\tau_z} + (I_x - I_y) \dot{\phi} \dot{\theta}. \quad (2.12)$$

The nonlinear dynamic model can be linearized about the equilibrium hovering condition. This operating point corresponds to a stationary configuration in which the quadcopter remains at rest with no translational motion and no attitude deviation, such that the roll and pitch angles satisfy $\phi \simeq 0$ and $\theta \simeq 0$, while the yaw angle ψ is constant. Without loss of generality, the yaw angle is set to $\psi = 0$ for simplicity. Under these assumptions, the small-angle approximation is applied, yielding $\cos(\phi) \simeq 1$, $\cos(\theta) \simeq 1$, $\sin(\phi) \simeq \phi$, and $\sin(\theta) \simeq \theta$. Furthermore, since the quadcopter is assumed to be in a steady hover, all angular velocities are neglected, *i.e.*, $\dot{\phi} = \dot{\theta} = \dot{\psi} = 0$, and the thrust forces generated by the four rotors are perfectly balanced. As a result, each rotor produces an equal thrust given by $F_1 = F_2 = F_3 = F_4 = mg/4$. Under these conditions, the system dynamics reduce to a linearized model, which can be expressed as:

$$\ddot{x} = \frac{F}{m} \theta, \quad (2.13)$$

$$\ddot{y} = \frac{F}{m} \phi, \quad (2.14)$$

$$\ddot{z} = \frac{F}{m} - g, \quad (2.15)$$

$$\ddot{\phi} = \frac{\tau_x}{I_x}, \quad (2.16)$$

$$\ddot{\theta} = \frac{\tau_y}{I_y}, \quad (2.17)$$

$$\ddot{\psi} = \frac{\tau_z}{I_z}. \quad (2.18)$$

A 12-dimensional state-space representation is considered, with the state vector defined as $\mathbf{x} = [x \ \dot{x} \ y \ \dot{y} \ z \ \dot{z} \ \phi \ \dot{\phi} \ \theta \ \dot{\theta} \ \psi \ \dot{\psi}]^\top$, which includes the translational positions and velocities along the three spatial axes, as well as the roll, pitch, and yaw angles and their corresponding angular velocities. The control input vector is defined as $\mathbf{u} = [F \ \tau_x \ \tau_y \ \tau_z]^\top$, where F denotes the total thrust generated by the four rotors and τ_x , τ_y , and τ_z represent the control torques about the body-fixed x , y , and z axes, respectively.

Under these definitions, the state-space model describing the linearized quadcopter dynamics can be written as:

$$\underbrace{\begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{y} \\ \ddot{y} \\ \dot{z} \\ \ddot{z} \\ \dot{\phi} \\ \ddot{\phi} \\ \dot{\theta} \\ \ddot{\theta} \\ \dot{\psi} \\ \ddot{\psi} \end{bmatrix}}_{\dot{\mathbf{x}}} = \underbrace{\begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & g & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & g & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} x \\ \dot{x} \\ y \\ \dot{y} \\ z \\ \dot{z} \\ \phi \\ \dot{\phi} \\ \theta \\ \dot{\theta} \\ \psi \\ \dot{\psi} \end{bmatrix}}_{\mathbf{x}} + \underbrace{\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ \frac{1}{m} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & \frac{1}{I_x} & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{I_y} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{I_z} \end{bmatrix}}_{\mathbf{B}} \underbrace{\begin{bmatrix} F \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix}}_{\mathbf{u}}. \quad (2.19)$$

The output equation is given by $\mathbf{y} = \mathbf{I}_{12} \mathbf{x}$, which reflects the assumption that all state variables are directly observed.

In order to perform numerical simulations, the continuous-time quadcopter model must be discretized. For the linearized dynamics, a zero-order hold (ZOH) assumption on the control inputs is adopted, leading to the discrete-time state-space representation:

$$\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k, \quad (2.20)$$

where $\mathbf{A}_d$ and $\mathbf{B}_d$ denote the discrete-time system matrices obtained from the continuous-time model for a given sampling period T_s . The ZOH method is commonly used in digital control applications and is well suited to this setting, as the control inputs are assumed to remain constant over each sampling interval.

For the nonlinear dynamics, time discretization is carried out using a forward Euler integration scheme, yielding:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + T_s f(\mathbf{x}_k, \mathbf{u}_k), \quad (2.21)$$

where the function $f(\cdot)$ represents the nonlinear continuous-time dynamics defined by (2.7)–(2.12). The forward Euler method is chosen for its simplicity and computational efficiency, which are sufficient for simulation purposes given the relatively small sampling time ($T_s = 0.1$ s) considered in this work.

The sampling period T_s is selected to be small enough to accurately capture the quadcopter dynamics while ensuring numerical stability of the discretization schemes. Under these conditions, both the ZOH discretization of the linear model and the forward Euler integration of the nonlinear model provide adequate accuracy for control design.

2.2 Control problem

This section formulates the control problem addressed in this project. The objective is to design a control policy capable of stabilizing the quadcopter at the origin in the presence of external disturbances, while respecting physical input constraints. The resulting formulation provides the foundation for the application of model-agnostic meta-learning techniques introduced in the next section.

2.2.1 Disturbance modeling

In realistic flight conditions, the quadcopter is subject to external disturbances, such as wind, which primarily affect its translational motion. To account for these effects, an additive disturbance term is introduced into the system dynamics. For the discrete-time linearized model, the dynamics are expressed as:

$$\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k + \mathbf{w}_k, \quad (2.22)$$

while for the nonlinear model, the disturbed dynamics are given by:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + T_s f(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{w}_k. \quad (2.23)$$

The disturbance vector $\mathbf{w}_k$ is defined as

$$\mathbf{w}_k = [d_x \ 0 \ d_y \ 0 \ d_z \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0]^\top, \quad (2.24)$$

where d_x and d_y represent wind-induced disturbances acting along the horizontal axes and d_z represents the gravity. In this project, d_z is kept constant while d_x and d_y are modeled as independent Gaussian random variables,

$$d_x \sim \mathcal{N}(\mu_x, \sigma_x^2), \quad d_y \sim \mathcal{N}(\mu_y, \sigma_y^2), \quad d_z = 9.81 \text{ m/s}^2. \quad (2.25)$$

Although this disturbance model is deliberately simple, it captures the dominant effect of wind on quadcopter motion and is sufficient for evaluating the robustness and adaptation capabilities of learning-based controllers.

2.2.2 Neural network policy

The control objective is to regulate the quadcopter to the origin, corresponding to a hovering configuration with zero position, velocity, and attitude error, despite the presence of disturbances. Formally, the desired equilibrium is given by:

$$\mathbf{x}_k = \mathbf{0}, \quad \forall k. \quad (2.26)$$

To achieve this objective, a feedback control policy is introduced that maps the full system state to the control input. A neural network-based controller is employed to generate the control inputs $\mathbf{u}_{\text{NN},k}$ in closed loop. Assuming full state availability for simplicity, the policy is defined as

$$\mathbf{u}_{\text{NN},k} = \pi_{\boldsymbol{\theta}}(\mathbf{x}_k), \quad (2.27)$$

where $\pi(\cdot)$ denotes a parameterized policy implemented as a multilayer perceptron (MLP).

Prior to being processed by the neural network, the state vector is normalized using predefined scaling factors in order to account for the different magnitudes and physical units of the state components. The normalized state is then propagated through the network to produce a raw control output.

The network architecture, illustrated schematically in Figure 2.2, consists of two hidden layers with 8 neurons each, resulting in the following mapping: $\mathbb{R}^{12} \rightarrow \mathbb{R}^8 \rightarrow \mathbb{R}^8 \rightarrow \mathbb{R}^4$. This architecture contains a total of 212 weights, collected in the parameter vector $\boldsymbol{\theta} \in \mathbb{R}^{212}$. The network parameters are unconstrained, while the policy outputs are explicitly bounded through the choice of activation functions in order to satisfy physical input constraints.

The policy is trained in closed loop by repeatedly rolling out the system dynamics over a finite time horizon and minimizing a cumulative cost function that penalizes deviations from the desired equilibrium as well as excessive control effort. This formulation allows the neural network to implicitly learn a feedback control strategy that compensates for disturbances and stabilizes the quadcopter.

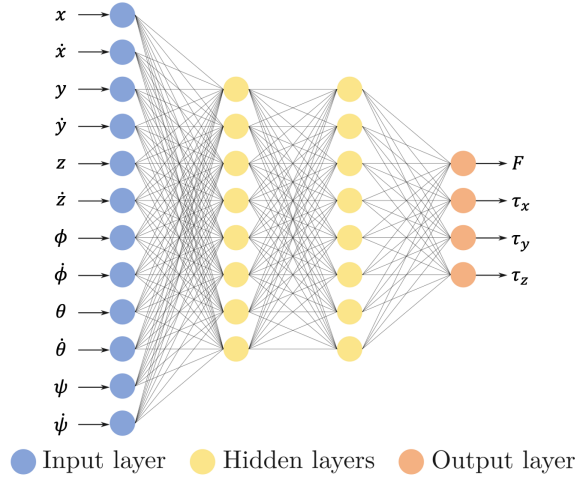


Figure 2.2: Schematic representation of a multilayer perceptron, with 12 input neurons, 2 hidden layers containing 8 neurons each, and 4 output neurons.

2.2.3 Cost function and optimization problem

The performance of a given policy is evaluated over a finite time horizon T using a quadratic cost function of the form:

$$\mathcal{L} = \sum_{k=0}^T \left(\mathbf{x}_k^\top Q \mathbf{x}_k + \mathbf{u}_k^\top R \mathbf{u}_k \right), \quad (2.28)$$

where $Q \succeq 0$ and $R \succ 0$ are weighting matrices penalizing state deviations and control effort, respectively. The diagonal elements of these matrices are chosen according to:

$$Q_{ii} = \frac{1}{x_{i,\max}^2}, \quad R_{jj} = \frac{1}{u_{j,\max}^2}, \quad (2.29)$$

where $x_{i,\max}$ and $u_{j,\max}$ denote admissible bounds on the states and inputs. This normalization provides an initial choice for the weights, which are subsequently refined through manual tuning depending on the task.

To make the simulation somewhat realistic, the control inputs are subject to physical constraints given by:

$$0 \leq F \leq F_{\max}, \quad (2.30)$$

$$-\tau_{\max} \leq \tau_x, \tau_y, \tau_z \leq \tau_{\max}. \quad (2.31)$$

The resulting control problem can be formulated as the following constrained optimization problem:

$$\begin{aligned} \min_{\boldsymbol{\theta} \in \mathbb{R}^{212}} \quad & \sum_{k=0}^T \left(\mathbf{x}_k^\top Q \mathbf{x}_k + \mathbf{u}_{\text{NN},k}^\top R \mathbf{u}_{\text{NN},k} \right) \\ \text{s.t.} \quad & \mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_{\text{NN},k} + \mathbf{w}_k, \\ & \mathbf{u}_{\text{NN},k} = \pi_{\boldsymbol{\theta}}(\mathbf{x}_k), \\ & 0 \leq F \leq F_{\max}, \\ & -\tau_{\max} \leq \tau_x, \tau_y, \tau_z \leq \tau_{\max}, \end{aligned} \quad (2.32)$$

where the disturbance sequence $\{\mathbf{w}_k\}_{k=0}^T$ is treated as an exogenous stochastic process and is sampled independently during training and evaluation.

3. Model-Agnostic Meta-Learning

This chapter introduces the meta-learning framework employed in this work to enable rapid adaptation of the quadcopter controller to changing wind conditions. After introducing the general concept of meta-learning and reviewing its main methodological families, the focus is placed on optimization-based approaches and, in particular, on Model-Agnostic Meta-Learning (MAML). The MAML algorithm is then presented and specialized to the quadcopter control problem, providing the basis for the numerical experiments reported in the following chapter.

3.1 Meta-learning background and related work

Meta-learning, often referred to as *learning to learn*, aims at training models that can rapidly adapt to new tasks using only a small amount of data. Rather than optimizing performance on a single task, meta-learning methods exploit a distribution of related tasks in order to acquire transferable knowledge that facilitates fast adaptation when confronted with previously unseen conditions. This paradigm is particularly well suited to control problems involving uncertainty or changing environments, where retraining a controller from scratch for each new scenario would be very computationally expensive.

Several families of meta-learning approaches have been proposed in the literature. *Model-based* methods embed the learning process directly into the model dynamics, typically through recurrent architectures or external memory, allowing rapid adaptation without explicit gradient updates. *Metric-based* approaches instead learn an embedding space in which new samples can be efficiently compared to previously seen examples, and are primarily used for few-shot classification or regression tasks. Finally, *optimization-based* methods aim to learn how to optimize by identifying parameter initializations or update rules that enable fast gradient-based adaptation on new tasks.

In addition to the general development of meta-learning algorithms, recent research has explored their application to aerial vehicle control under varying environmental conditions. For example, O’Connell *et al.* proposed a meta-learning-based robust adaptive flight controller that combines deep neural network basis functions with online adaptation to handle uncertain wind conditions, and demonstrated improved trajectory tracking in wind tunnel experiments [4]. Their approach uses meta-learning to pretrain representations useful for adaptation, while adapting only a subset of parameters online, enabling real-time deployment with limited computational overhead. A related line of work, named Neural-Fly [5], employs domain-invariant meta representations of aerodynamic effects to achieve rapid online adaptation and robust performance in strong wind conditions. While these methods demonstrate the effectiveness of meta-learning for real-world wind disturbance rejection, they primarily focus on learning representations or basis functions for adaptive control rather than directly optimizing a full feedback policy. In contrast to these approaches, the present work investigates the use of Model-Agnostic Meta-Learning (MAML) to directly optimize the parameters of a full neural network feedback controller. Rather than restricting adaptation to predefined basis functions or a subset of parameters, the proposed method learns an initialization that enables fast gradient-based adaptation of the entire control policy to previously unseen wind conditions.

3.2 Optimization-based meta-learning and MAML

Among optimization-based approaches, Model-Agnostic Meta-Learning (MAML), introduced by Finn *et al.* [6], has emerged as a general and widely applicable framework. The core idea of MAML is to learn a set of model parameters that serves as a good initialization, such that only a small number of gradient descent steps are required to achieve good performance on a new task sampled from the same

task distribution. A key advantage of MAML is its model-agnostic nature: it can be applied to any learning problem that can be trained using gradient-based optimization, including supervised learning, reinforcement learning, and control.

Formally, MAML considers a distribution over tasks, where each task corresponds to a loss function. During meta-training, tasks are sampled from this distribution, and for each task an inner-loop optimization is performed using a small dataset to obtain task-adapted parameters. The meta-objective is then defined as the performance of these adapted parameters on new data from the same task, and gradients are propagated through the inner-loop updates to optimize the shared initialization. This process explicitly optimizes for fast within-task generalization, rather than for average performance across tasks.

Despite its strong empirical performance, MAML requires computing second-order derivatives during the meta-update, which can be computationally expensive. To address this limitation, first-order variants have been proposed. In particular, Reptile [7] avoids explicit second-order gradients by moving the initialization toward parameters that perform well after task-specific training. While simpler to implement and more scalable, such first-order methods may yield less precise adaptation than full MAML in some settings. Another extension of MAML is PLATIPUS [8], which incorporates uncertainty into the meta-learning process by learning a distribution over initial parameters rather than a single point estimate. This Bayesian formulation enables more robust adaptation in stochastic or noisy environments, at the cost of increased algorithmic complexity. In this work, standard MAML is adopted as a well-established and theoretically grounded reference method that explicitly optimizes for fast gradient-based adaptation while remaining computationally manageable within the scope of simulation-based quadcopter control experiments.

3.3 MAML algorithm

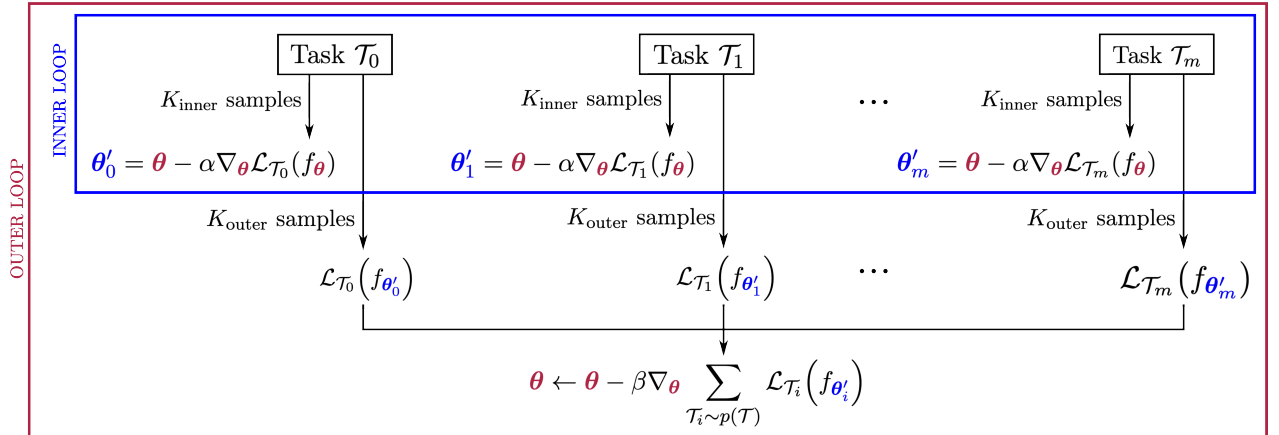
This section presents the Model-Agnostic Meta-Learning (MAML) algorithm employed in this project and describes how it is applied for the quadcopter control problem.

Figure 3.1 illustrates a schematic overview of the original MAML procedure as introduced by Finn *et al.* [6]. MAML considers a distribution of tasks $p(\mathcal{T})$, where each task $\mathcal{T}_i$ is associated with a task-specific loss function $\mathcal{L}_{\mathcal{T}_i}(\cdot)$ (i.e. (2.28) in this project). The objective is to learn a set of parameters θ such that a small number of gradient updates yields good performance on a previously unseen task sampled from the same distribution.

m tasks $\mathcal{T}_i \sim p(\mathcal{T})$

Randomly initialize θ

While not done **do**



end While

Figure 3.1: Schematic overview of the Model-Agnostic Meta-Learning (MAML) algorithm.

3.3.1 Meta-training procedure

During meta-training, m tasks $\mathcal{T}_i \sim p(\mathcal{T})$ are sampled. The model parameters θ are initialized randomly at the beginning of the training and subsequently optimized.

For each task $\mathcal{T}_i$, an inner loop update is performed using a small dataset of K_{inner} samples drawn from that task. This dataset is intentionally kept small in order to explicitly train the model for rapid adaptation under limited data availability. The task-adapted parameters are obtained via a single gradient descent step of the form:

$$\theta'_i = \theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta}), \quad (3.1)$$

where α denotes the inner loop learning rate and f_{θ} represents the neural network controller parameterized by θ .

After computing the adapted parameters θ'_i for each task, an outer-loop (meta) update is performed. For this update, a second dataset of K_{outer} samples is drawn from each task, different from the inner loop data. The meta-objective then minimizes the sum of the losses evaluated using the task-adapted parameters:

$$\theta \leftarrow \theta - \beta \nabla_{\theta} \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i}(f_{\theta'_i}), \quad (3.2)$$

where β denotes the outer loop learning rate. This update explicitly optimizes the initial parameters θ such that they can be rapidly adapted to any task drawn from $p(\mathcal{T})$.

After convergence of the meta-training process, the resulting parameters θ constitute a meta-learned initialization that is suitable for few-shot adaptation.

3.3.2 Few-shot adaptation on a new task

At testing, a new and previously unseen task $\mathcal{T}_{\text{target}} \sim p(\mathcal{T})$ is considered. The parameters are initialized using the meta-learned initialization θ obtained at the end of meta-training. Adaptation to the new task is then performed by applying a small number of gradient descent steps using K_{inner} samples drawn from $\mathcal{T}_{\text{target}}$:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_{\text{target}}}(f_{\theta}). \quad (3.3)$$

The resulting parameters are expected to yield strong performance on the target task despite the limited amount of task-specific data.

3.3.3 Baseline: joint training

For the numerical results presented in the following chapter, the performance of the meta-learned controller is compared against a baseline obtained through joint training across all tasks. In this baseline approach, wind disturbance sequences from all tasks are pooled together and the neural network controller is trained using standard gradient descent until convergence of the training loss.

This corresponds to a naive strategy in which all available data are used to train a single controller without explicitly accounting for task structure or adaptation.

To ensure a fair comparison, few-shot adaptation is also applied to this baseline by performing a small number of additional gradient updates (3.3) using data from the target task (only K_{inner} samples drawn from $\mathcal{T}_{\text{target}}$), starting from the jointly trained parameters. This allows a direct comparison between meta-learning and conventional training in terms of adaptation speed and performance on previously unseen wind conditions.

3.3.4 Task definition for quadcopter control

In this project, each task corresponds to a specific wind disturbance configuration acting on the quadcopter system. A total of m tasks is considered, indexed by $i \in \{1, \dots, m\}$. Each task $\mathcal{T}_i$ is defined

as:

$$\mathcal{T}_i = (\mathcal{N}(\mu_i, \sigma_i^2), \gamma_i), \quad (3.4)$$

where γ_i denotes a fixed wind direction in the horizontal plane and $\mathcal{N}(\mu_i, \sigma_i^2)$ corresponds to the wind amplitude distribution.

For each task, a batch of trajectories is generated by simulating multiple quadcopters over a finite time horizon T . Each trajectory experiences a time-varying wind disturbance whose amplitude is independently sampled at each time step from the corresponding normal distribution. Formally, the wind-induced disturbances along the x and y axes are given by

$$\begin{aligned} \{d_x\}_{k=0}^T &= A_k \cos(\gamma_i), & A_k &\sim \mathcal{N}(\mu_i, \sigma_i^2), \\ \{d_y\}_{k=0}^T &= A_k \sin(\gamma_i), & A_k &\sim \mathcal{N}(\mu_i, \sigma_i^2), \end{aligned} \quad (3.5)$$

where A_k denotes the wind amplitude at time step k . As a result, all trajectories within the same task share the same wind direction and statistical amplitude profile, while exhibiting different realizations of the disturbance over time.

4. Numerical Results

This chapter presents the numerical results obtained throughout the project. A sanity check is first conducted to validate the behavior of the neural network-based controller. A set of motivation experiments is then introduced to highlight the relevance of meta-learning for quadcopter control under wind disturbances. Finally, the results of MAML experiments are reported under increasing levels of difficulty: wind disturbances acting exclusively along the x -axis and from all directions, first using linearized dynamics and then the full nonlinear model.

4.1 Neural network sanity check

Before investigating meta-learning strategies, a preliminary sanity check is conducted to verify that the neural network policy (2.27) yields a meaningful and effective controller. To this end, a simple regulation task is considered and the neural network controller $\mathbf{u}_{\text{NN},k}$ is compared against a classical Linear Quadratic Regulator (LQR), denoted $\mathbf{u}_{\text{LQR},k}$. The control objective is to regulate the agent to the origin. Here, the linearized state-space model (2.19) is used.

The LQR controller is obtained by solving the discrete algebraic Riccati equation (DARE) and computing the associated optimal feedback gain (see Appendix 5.1). The neural network is trained using 27 agents whose initial positions are chosen to best describe the cube $(-5, 5)^3$ m, including the vertices, face centers, edge centers, and the cube center. Figure 4.1 illustrates the corresponding initial positions.

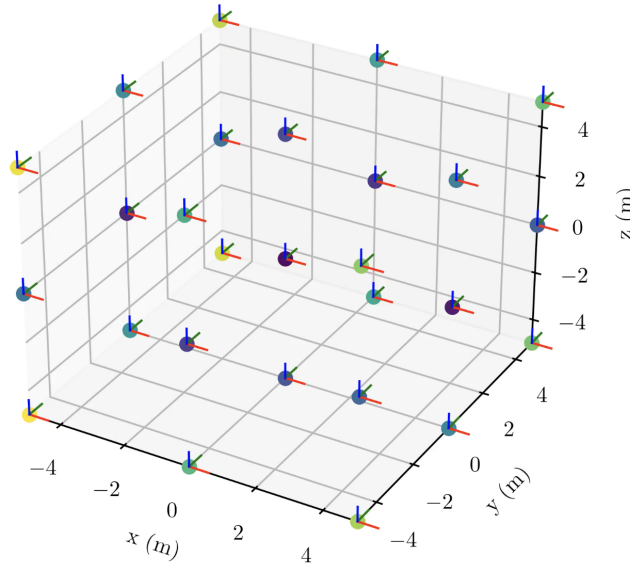
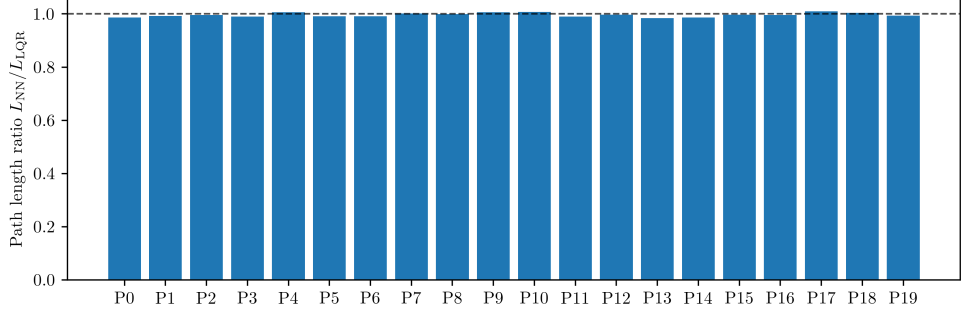
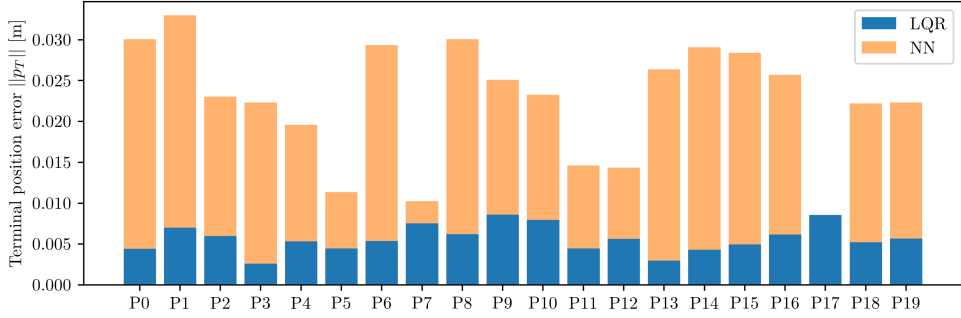


Figure 4.1: Initial positions of the 27 agents used to train the neural network controller for the sanity check.

Both controllers are then evaluated using 20 additional agents with randomly sampled initial positions within the same cube. In both cases, the same quadratic cost function (2.28), with identical weighting matrices Q and R , is used. The results show that both controllers exhibit very similar behavior. Figure 4.2a indicates that the trajectory lengths are nearly identical for all agents, suggesting highly comparable paths. Figure 4.2b shows that although the neural network controller generally yields a slightly larger terminal error than the LQR, the errors remain small and acceptable.



(a) Ratio of trajectory lengths produced by the neural network and LQR controllers for 20 randomly initialized agents.



(b) Terminal position error of each of the 20 randomly initialized agents under neural network and LQR control.

Figure 4.2: Comparison of regulation performance between the neural network controller and the LQR controller over 20 randomly initialized agents.

Overall, this experiment confirms that the neural network controller successfully reproduces the behavior of a well-designed linear controller, thereby validating its use in the subsequent experiments.

4.2 Motivation experiments

Having validated the neural network controller on a simple regulation task, this section aims to motivate the use of meta-learning before introducing the MAML algorithm. The core idea of meta-learning is to exploit the structure and similarity between related tasks, here different wind conditions, in order to achieve rapid adaptation from limited data. In contrast, standard training approaches require retraining from scratch for each new disturbance, which quickly becomes prohibitive in terms of data and computational cost. To illustrate this limitation, two wind disturbance tasks are considered: $\mathcal{T}_1 = (\mathcal{N}(5, 1), 0^\circ)$, $\mathcal{T}_2 = (\mathcal{N}(5, 1), 180^\circ)$ (corresponding to wind distributions with identical magnitude and variance but opposite directions). The following question is investigated: if abundant data are available for one wind condition (500 trajectories) but only a few samples for another wind condition (5 trajectories), can a standard neural network controller generalize successfully without meta-learning? To answer this question, the following three training configurations are evaluated:

$$\text{Training case 1: } 500 \text{ agents, } \{d_x\}_{k=0}^T = A_k, A_k \sim \mathcal{N}(5, 1), \{d_y\}_{k=0}^T = 0 \quad (4.1a)$$

$$\text{Training case 2: } 5 \text{ agents, } \{d_x\}_{k=0}^T = -A_k, A_k \sim \mathcal{N}(5, 1), \{d_y\}_{k=0}^T = 0 \quad (4.1b)$$

$$\text{Training case 3: } \text{Training case 1} \cup \text{Training case 2} \quad (4.1c)$$

After training, all controllers are evaluated on the full second task distribution:

$$\text{Testing: } 500 \text{ agents, } \{d_x\}_{k=0}^T = -A_k, A_k \sim \mathcal{N}(5, 1), \{d_y\}_{k=0}^T = 0 \quad (4.2)$$

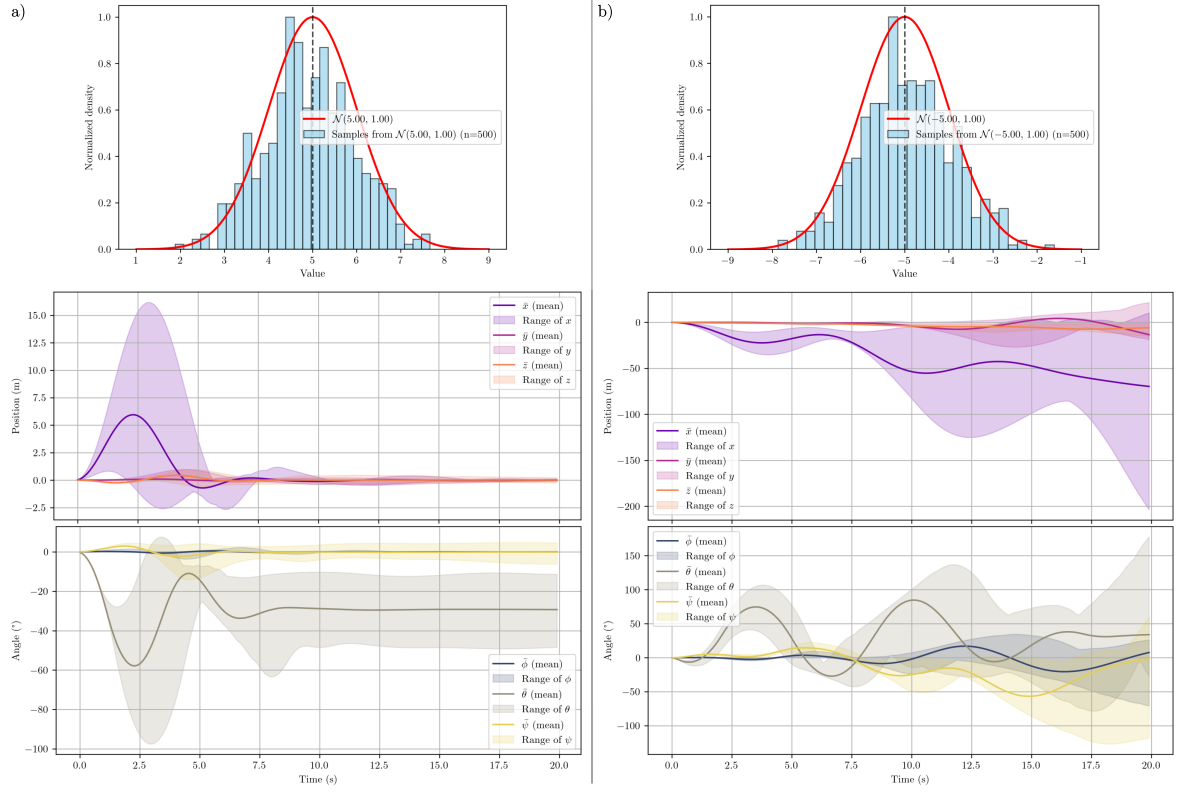


Figure 4.3: Motivation experiment – Case 1. **a)** Training on wind task $\mathcal{T}_1$ using 500 agents (4.1a). **b)** Testing on the full wind task $\mathcal{T}_2$ (4.2).

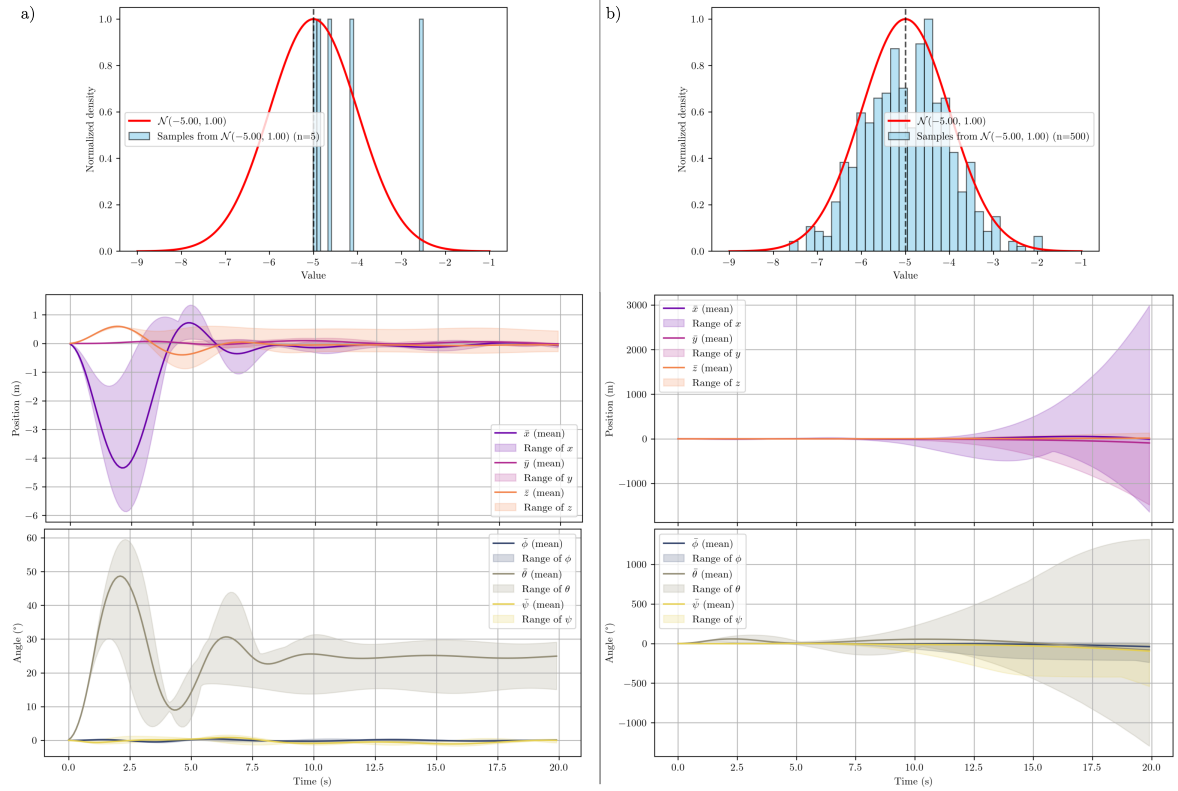


Figure 4.4: Motivation experiment – Case 2. **a)** Training on wind task $\mathcal{T}_2$ using only 5 agents (4.1b). **b)** Testing on the full wind task $\mathcal{T}_2$ (4.2).

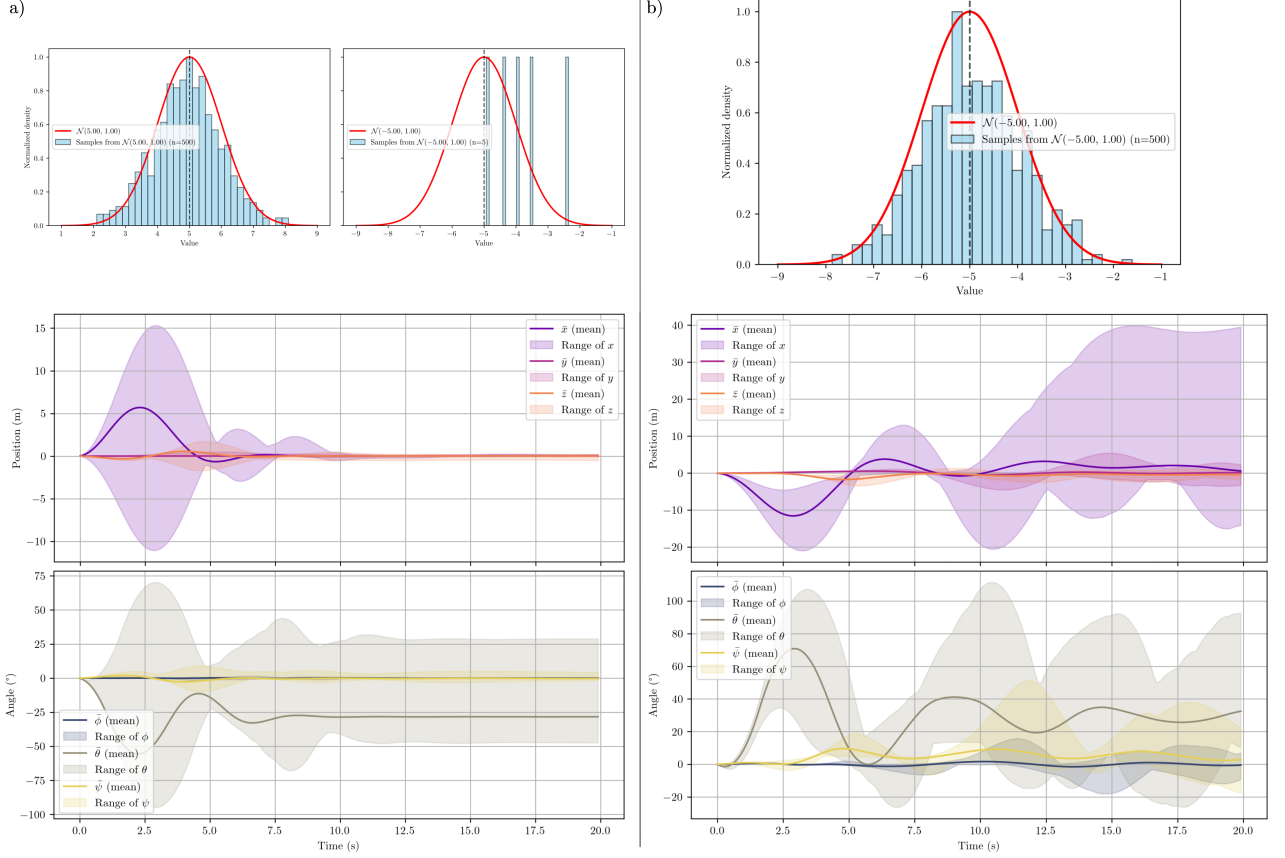


Figure 4.5: Motivation experiment – Case 3. **a)** Training on $\mathcal{T}_1$ (500 agents) and $\mathcal{T}_2$ (5 agents) (4.1c). **b)** Testing on the full wind task $\mathcal{T}_2$ (4.2).

In all motivation experiments, the agents are initialized at the origin and are required to reject the wind disturbance so as to remain at the origin. All simulations are conducted using the linearized state-space model (2.19).

Across all three cases, the neural network controller fails to achieve satisfactory performance over the full task distribution $\mathcal{T}_2$. First, as illustrated in Figure 4.3, when the controller is trained exclusively on samples from $\mathcal{T}_1$, it performs very poorly on samples drawn from $\mathcal{T}_2$. This behavior is expected, as no information about the second wind condition is available during training. Second, when only a few samples from $\mathcal{T}_2$ are included during training, Figure 4.4 shows that the controller still fails to generalize across the entire distribution of $\mathcal{T}_2$. Finally, Figure 4.5 demonstrates that even when combining a large number of samples from $\mathcal{T}_1$ with the limited available samples from $\mathcal{T}_2$, the resulting controller does not generalize satisfactorily to unseen realizations of $\mathcal{T}_2$.

To further quantify the lack of generalization observed in the motivation experiments, the mean cost is evaluated on both a validation set (drawn from the training distribution) and a testing set (drawn from the testing distribution).

Table 4.1: Validation and testing performance for the motivation experiments.

Experiment	$\mathbb{E}[J_{\text{val.}}]$	$\mathbb{E}[J_{\text{test.}}]$	Increase (%)
Case 1	2.9×10^2	2.6×10^5	9.27×10^4
Case 2	1.5×10^2	1.8×10^7	1.21×10^7
Case 3	2.9×10^2	3.0×10^3	9.88×10^2

Table 4.1 quantitatively confirms the conclusions drawn from the qualitative analysis. In all three cases, the testing cost on the testing distribution is several orders of magnitude larger than the validation cost, highlighting a severe generalization gap. Even when limited samples from $\mathcal{T}_2$ are included during training (Case 3), the controller remains unable to generalize reliably to unseen realizations of the same wind condition. Overall, these results indicate that standard training approaches struggle when data availability is highly unbalanced across tasks, and they motivate the use of meta-learning techniques specifically designed for rapid adaptation from few samples.

4.3 MAML experiments with linearized dynamics

This section investigates the use of the Model-Agnostic Meta-Learning (MAML) algorithm, introduced in Section 3.3, for wind disturbance rejection using the linearized state-space model (2.19). The difficulty of the considered task distributions is progressively increased. First, wind disturbances are restricted to a single axis. Second, wind disturbances are allowed to come from all directions. Finally, the robustness of the few-shot adaptation in the MAML algorithm is studied.

4.3.1 Wind disturbance: single-axis

The first meta-learning experiment considers wind disturbances acting exclusively along the x -axis, with $\{d_y\}_{k=0}^T = 0$ for all tasks. As illustrated in Figure 4.6, four training tasks are defined. Three tasks correspond to wind disturbances originating from the negative direction of the axis, while the fourth task corresponds to a disturbance from the positive direction. The target task $\mathcal{T}_{\text{target}}$ involves a high-amplitude wind disturbance acting along the positive x -axis and is not included during training.

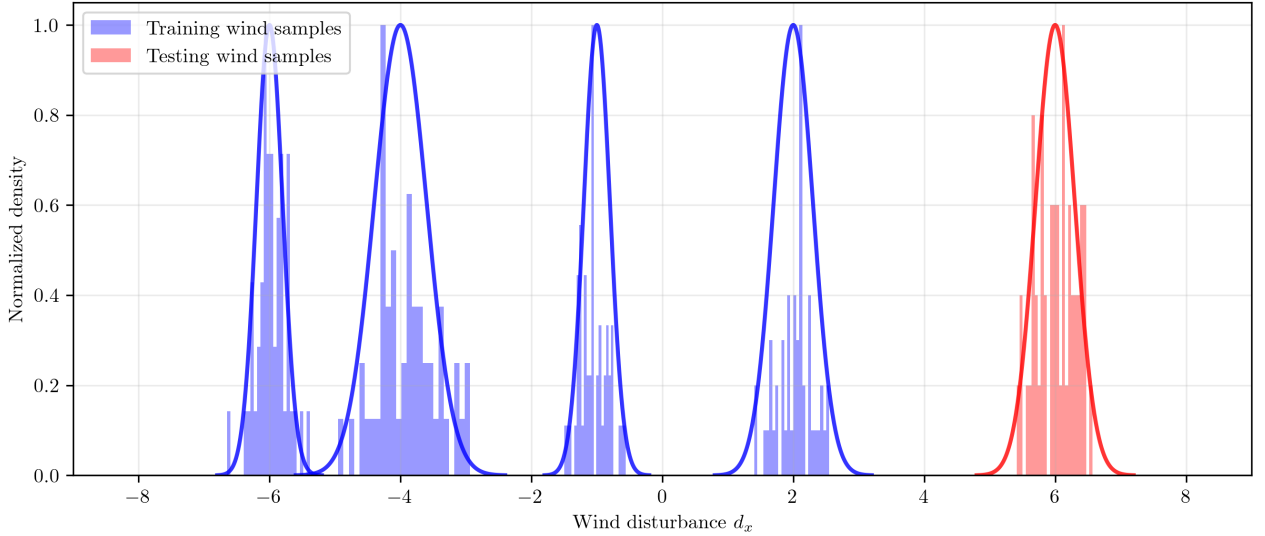


Figure 4.6: Training and testing wind disturbance distributions for the single-axis experiment ($\mathcal{T}_1 = (\mathcal{N}(6, 0.2), 180^\circ)$, $\mathcal{T}_2 = (\mathcal{N}(4, 0.4), 180^\circ)$, $\mathcal{T}_3 = (\mathcal{N}(1, 0.2), 180^\circ)$, $\mathcal{T}_4 = (\mathcal{N}(2, 0.3), 0^\circ)$, and $\mathcal{T}_{\text{target}} = (\mathcal{N}(6, 0.3), 0^\circ)$).

To establish a baseline for comparison against which the performance of the meta-learned controller is later compared, a joint training approach is first computed. In this approach, samples from all four training tasks are pooled together and the neural network controller is trained using standard gradient descent until convergence of the training loss, whose evolution is shown in Figure 4.7.

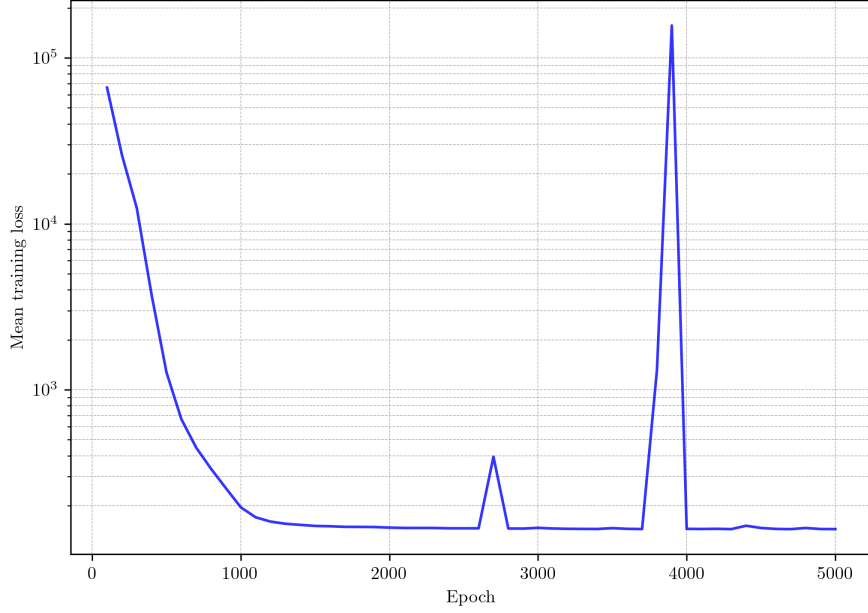


Figure 4.7: Single-axis experiment: evolution of the joint training loss across all four training tasks.

The meta-training process is illustrated in Figure 4.8. Figure 4.8a presents the inner-loop losses associated with each training task, while Figure 4.8b shows the evolution of the meta (outer-loop) loss. It can be observed that the meta loss converges to a loss level comparable to that obtained through joint training, indicating similar performance on the training tasks.

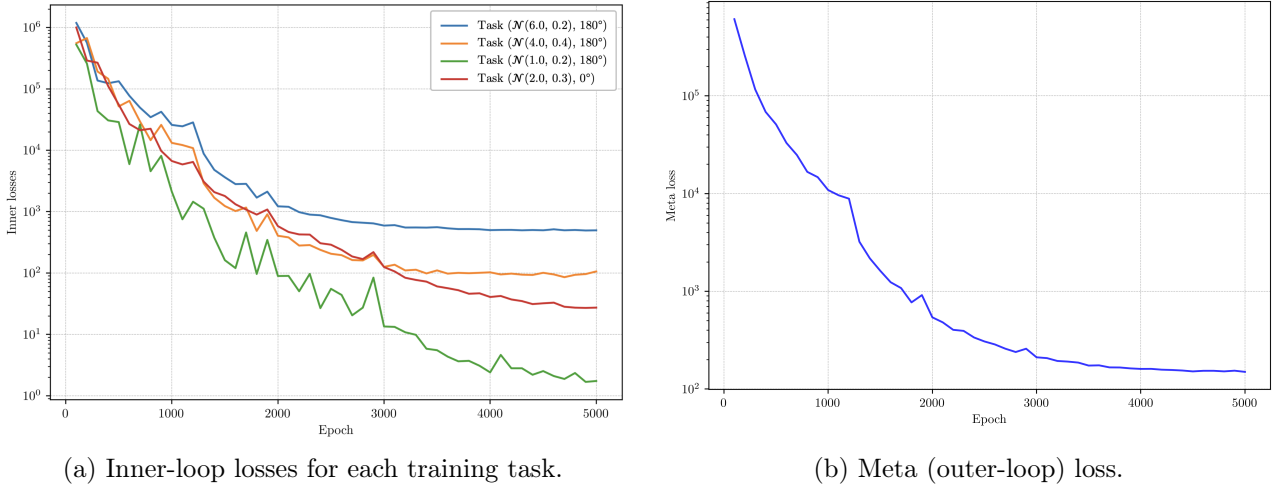


Figure 4.8: Single-axis experiment: meta-training losses.

Once the joint-training and meta-training phases are completed, few-shot adaptation to the unseen target task $\mathcal{T}_{\text{target}}$ is performed. Starting from the parameters obtained after joint training and meta-training, respectively, five gradient update steps are applied. Each update uses five randomly sampled trajectories drawn from $\mathcal{T}_{\text{target}}$.

After each adaptation step, the updated controller is evaluated on the full target task distribution. Figure 4.9 compares the resulting losses on $\mathcal{T}_{\text{target}}$ for the joint-trained and meta-trained initializations throughout the adaptation process.

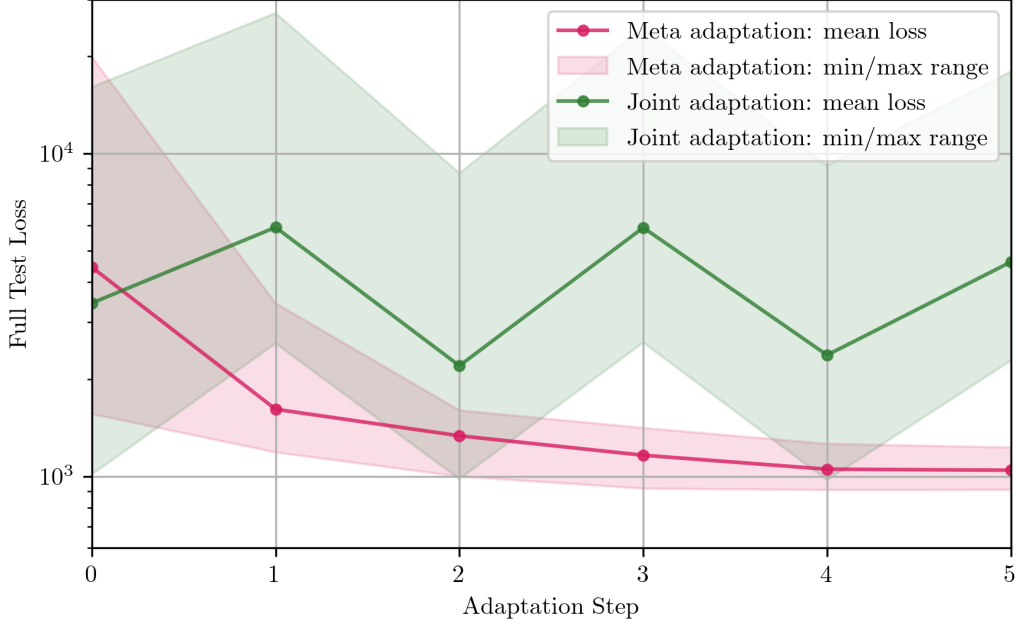


Figure 4.9: Single-axis experiment: comparison of few-shot adaptation performance on the target task $\mathcal{T}_{\text{target}}$, starting from joint-trained and meta-trained initializations.

Several observations can be drawn from these results. First, before any adaptation step is performed, the initial loss on $\mathcal{T}_{\text{target}}$ is higher for the meta-trained parameters than for the jointly trained parameters. This behavior is expected: the meta-trained initialization is explicitly optimized for fast adaptation rather than immediate performance on unseen tasks, whereas the jointly trained parameters can be interpreted as an average solution over the training tasks and therefore perform reasonably well on related but unseen conditions.

More importantly, Figure 4.9 shows that, after only a few adaptation steps, the loss associated with the meta-trained initialization decreases rapidly and monotonically. In contrast, the loss obtained from the jointly trained initialization exhibits oscillatory behavior and does not show a clear downward trend. This indicates that joint training does not provide a parameter initialization that is well suited for rapid adaptation to the target task.

To further illustrate this difference in behavior, Figure 4.10 presents quadcopter trajectories subject to wind disturbances sampled from $\mathcal{T}_{\text{target}}$, using the parameters obtained after five adaptation steps. When starting from the meta-trained initialization, the controller successfully rejects the disturbance and maintains the agent close to the origin (*i.e.* Figure 4.10a). In contrast, the controller initialized from jointly trained parameters fails to compensate for the disturbance, leading to diverging trajectories (*i.e.* Figure 4.10b).

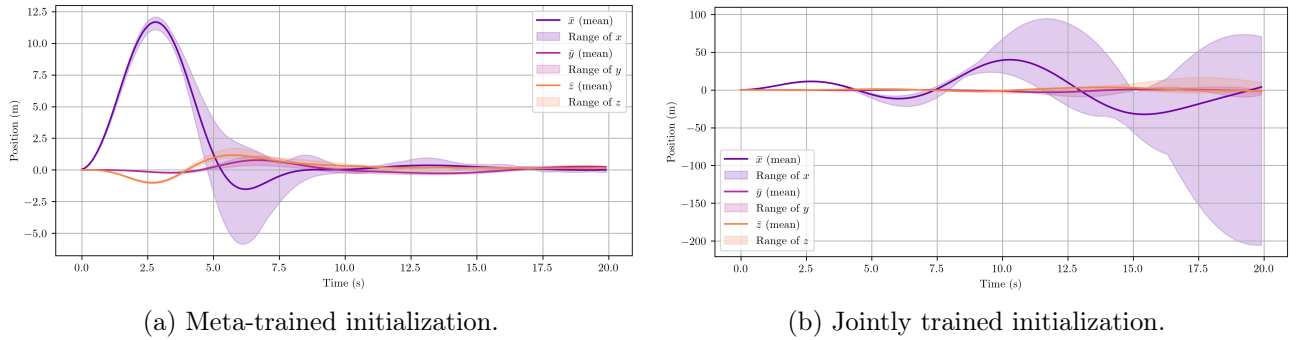


Figure 4.10: Single-axis experiment: simulated quadcopter trajectories under wind disturbances sampled from $\mathcal{T}_{\text{target}}$, using parameters obtained after five adaptation steps.

4.3.2 Wind disturbance: all directions

In this second meta-learning experiment, both the wind direction and amplitude are randomly generated, resulting in disturbance components along both the x - and y -axes. Compared to the single-axis case, this setting introduces a substantially more challenging task distribution.

As illustrated in Figure 4.11, six training tasks are defined, whose wind directions lie in the first, second, and fourth quadrants of the unit circle. The target task $\mathcal{T}_{\text{target}}$ is located in the third quadrant and is not included during training. This configuration is designed to deliberately exclude an entire range of wind directions during training, thereby testing the ability of the controller to adapt to previously unseen directional disturbances.

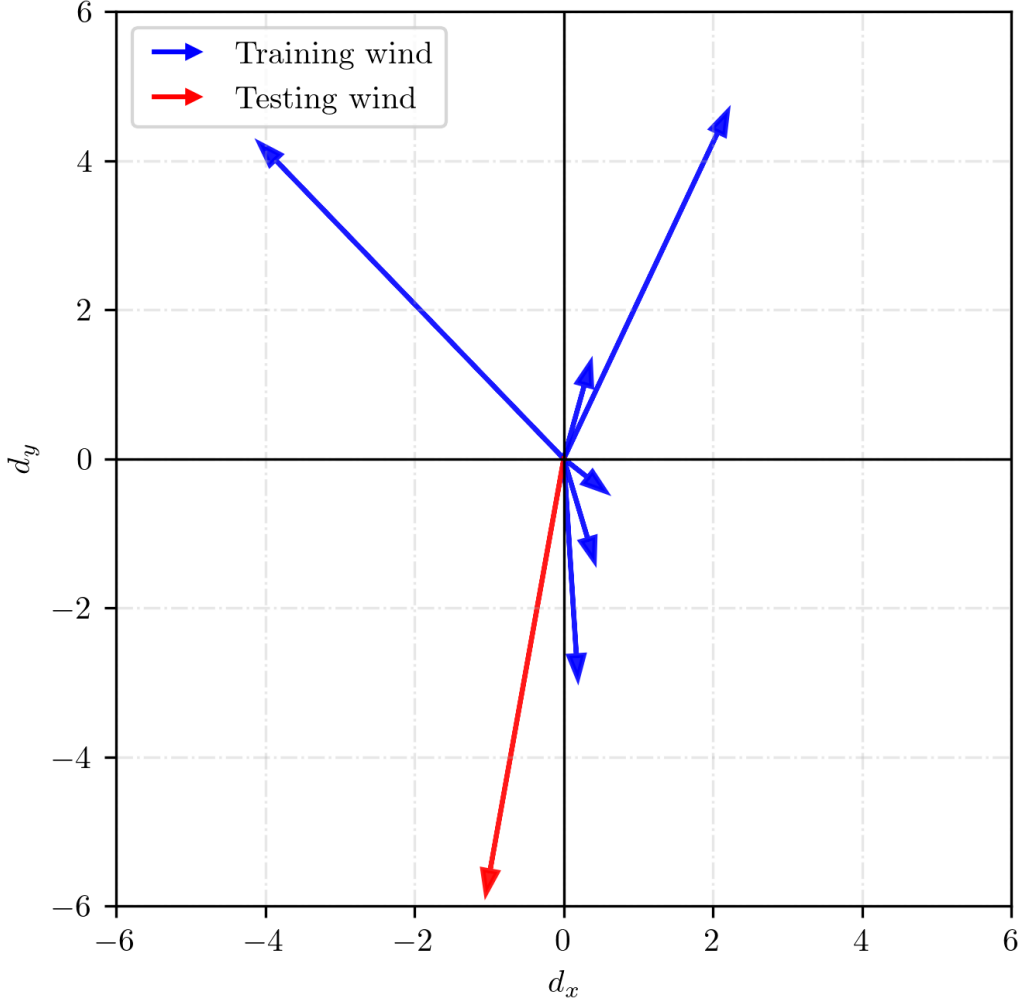


Figure 4.11: Training and testing wind disturbance distributions for the all-directions experiment ($\mathcal{T}_1 = (\mathcal{N}(1.3, 0.5), 74^\circ)$, $\mathcal{T}_2 = (\mathcal{N}(5.9, 0.3), 134^\circ)$, $\mathcal{T}_3 = (\mathcal{N}(1.4, 0.5), 287^\circ)$, $\mathcal{T}_4 = (\mathcal{N}(0.7, 0.1), 322^\circ)$, $\mathcal{T}_5 = (\mathcal{N}(3.0, 0.3), 274^\circ)$, $\mathcal{T}_6 = (\mathcal{N}(5.2, 0.4), 65^\circ)$, and $\mathcal{T}_{\text{target}} = (\mathcal{N}(5.9, 0.3), 260^\circ)$).

As in the previous experiment, a baseline is first established through joint training across all training tasks. In this approach, disturbance sequences from all training tasks are pooled together and the neural network controller is optimized using standard gradient descent. The evolution of the joint training loss is shown in Figure 4.12.

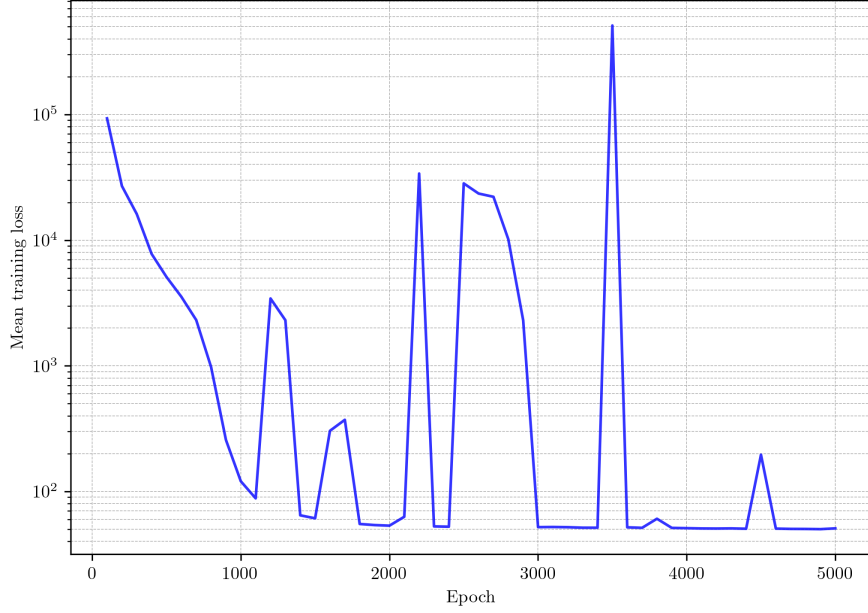


Figure 4.12: All-directions experiment: evolution of the joint training loss across all six training tasks.

The meta-training process is illustrated in Figure 4.13. Figure 4.13a shows the inner-loop losses associated with each training task, while Figure 4.13b presents the evolution of the meta (outer-loop) loss. As in the single-axis experiment, the meta loss converges to a level comparable to that obtained through joint training, indicating similar performance on the training tasks.

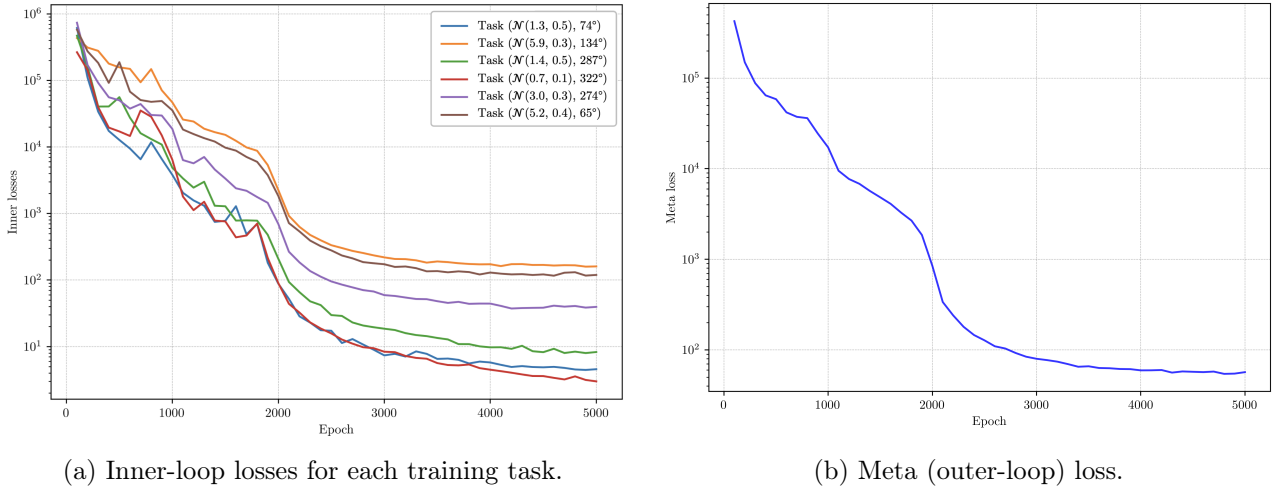


Figure 4.13: All-directions experiment: meta-training losses.

Once the joint-training and meta-training phases are completed, few-shot adaptation to the unseen target task $\mathcal{T}_{\text{target}}$ is performed. Starting from the parameters obtained after joint training and meta-training, respectively, five gradient update steps are applied, each using five randomly sampled trajectories drawn from $\mathcal{T}_{\text{target}}$.

After each adaptation step, the updated controller is evaluated on the full target task distribution. Figure 4.14 compares the resulting losses on $\mathcal{T}_{\text{target}}$ for the joint-trained and meta-trained initializations throughout the adaptation process.

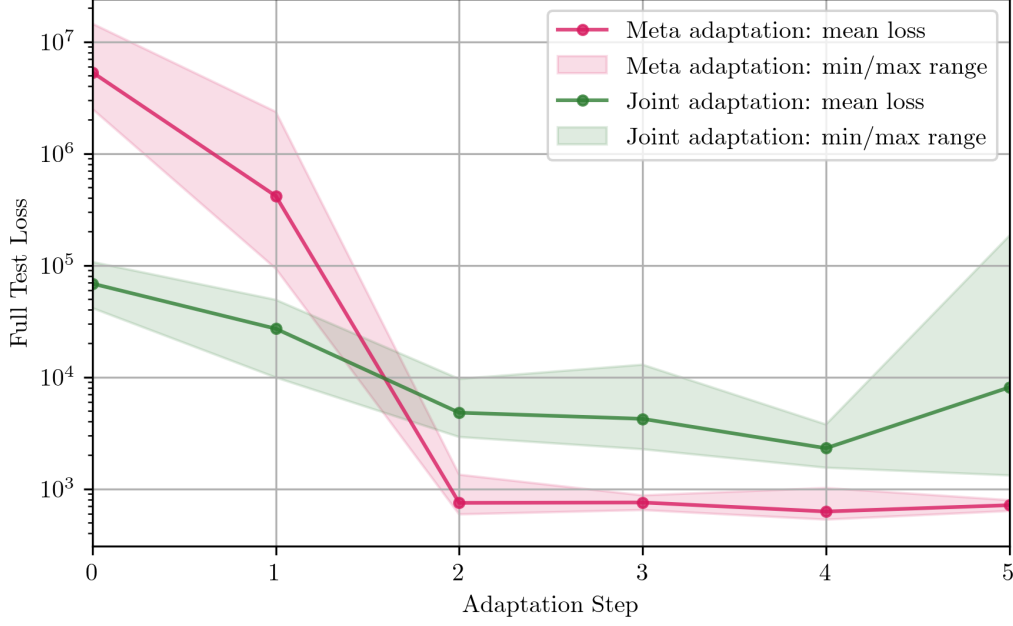


Figure 4.14: All-directions experiment: comparison of few-shot adaptation performance on the target task $\mathcal{T}_{\text{target}}$, starting from joint-trained and meta-trained initializations.

The same qualitative trends observed in the single-axis experiment also appear in this more challenging setting. Before any adaptation step is performed, the initial loss on $\mathcal{T}_{\text{target}}$ is higher for the meta-trained initialization than for the jointly trained one. However, Figure 4.14 shows that the loss associated with the meta-trained parameters decreases rapidly and monotonically over the first few adaptation steps before reaching a plateau. In contrast, the loss obtained from the jointly trained initialization decreases more slowly and exhibits a larger variability across the target task distribution. This indicates that, although joint training provides reasonable initial performance, it does not yield a parameter initialization that is well suited for rapid adaptation to unseen wind directions.

To further illustrate this difference in behavior, Figure 4.15 presents representative quadcopter trajectories subject to wind disturbances sampled from $\mathcal{T}_{\text{target}}$, using the parameters obtained after five adaptation steps. When starting from the meta-trained initialization, the controller successfully rejects the disturbance and maintains the agent close to the origin (Figure 4.15a). In contrast, the controller initialized from jointly trained parameters fails to compensate for the disturbance, leading to diverging trajectories (Figure 4.15b).

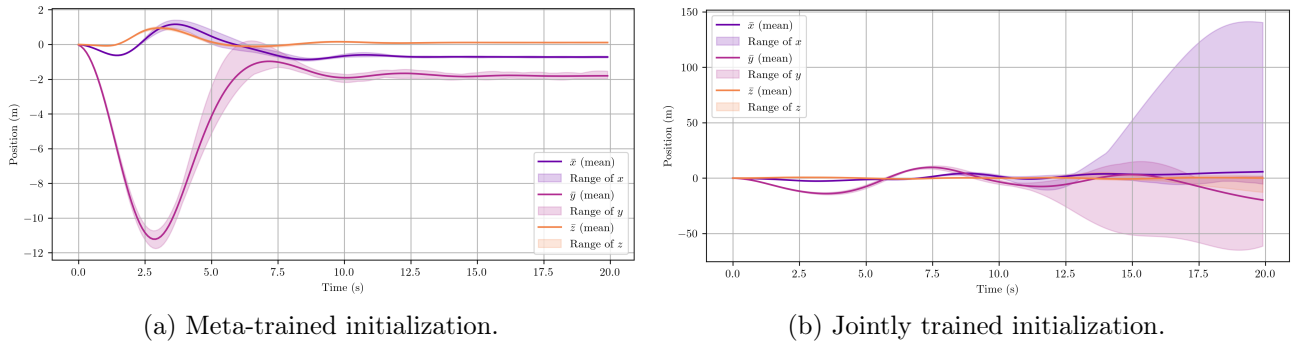


Figure 4.15: All-directions experiment: simulated quadcopter trajectories under wind disturbances sampled from $\mathcal{T}_{\text{target}}$, using parameters obtained after five adaptation steps.

4.3.3 Robustness analysis

In the previous meta-learning experiments (single-axis and all-directions), the few-shot adaptation process was performed using five randomly selected trajectories drawn from the target task distribution $\mathcal{T}_{\text{target}}$. Since the wind amplitude varies across samples due to the nonzero variance of the disturbance distribution, the particular choice of samples used during adaptation may influence the resulting performance.

To assess the robustness of the adaptation process with respect to this sample selection, the adaptation experiments from the previous sections are repeated ten times. Each repetition uses a different random subset of five trajectories drawn from $\mathcal{T}_{\text{target}}$. The resulting adaptation losses for both the single-axis and all-directions experiments are reported in Figure 4.16.

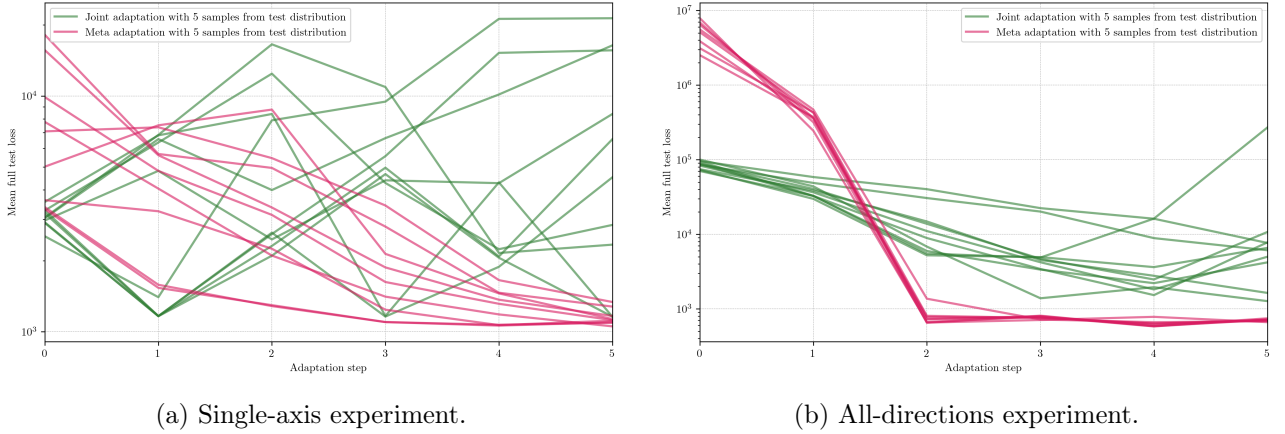


Figure 4.16: Robustness of the few-shot adaptation process. Comparison of adaptation performance on the target task $\mathcal{T}_{\text{target}}$ for both the single-axis and all-directions experiments, starting from joint-trained and meta-trained initializations. In each case, ten independent adaptation runs are performed using five randomly selected trajectories from $\mathcal{T}_{\text{target}}$.

In the single-axis experiment, shown in Figure 4.16a, all adaptation runs starting from the meta-trained initialization decrease rapidly and converge to a low loss after the five adaptation steps. In contrast, the adaptation trajectories obtained from the jointly trained initialization exhibit significantly higher variability, with losses oscillating over a wide range and failing to consistently converge to low values. This indicates that, in this setting, joint training leads to an adaptation process that is both less reliable and more sensitive to the specific choice of adaptation samples.

The all-directions experiment, illustrated in Figure 4.16b, exhibits an even more pronounced difference. For the meta-trained initialization, all adaptation runs decrease sharply after only two adaptation steps and converge to nearly identical low-loss values. The close clustering of the adaptation curves indicates that, in this more challenging setting, the performance of the meta-learned controller is largely insensitive to the particular samples drawn from $\mathcal{T}_{\text{target}}$. In contrast, while the adaptation curves associated with the jointly trained initialization are also relatively well clustered, their losses decrease much more slowly and remain approximately one order of magnitude higher than those obtained with the meta-trained initialization after five adaptation steps. This confirms that joint training does not provide a parameter initialization that supports effective or reliable few-shot adaptation.

Overall, this robustness analysis demonstrates that the meta-learned initialization not only enables rapid adaptation to unseen wind conditions, but also yields an adaptation process that is stable and largely insensitive to the specific choice of adaptation samples, in contrast to standard joint training approaches.

4.4 MAML experiments with nonlinear dynamics

This section investigates the use of the Model-Agnostic Meta-Learning (MAML) algorithm, introduced in Section 3.3, for wind disturbance rejection when considering the full nonlinear quadcopter dynamics (2.7)-(2.12). The objective is twofold. First, a simple regulation task without disturbances is used to compare controllers trained on linearized and nonlinear dynamics. Second, the meta-learning experiments presented in Section 4.3 are repeated using the nonlinear dynamics in order to assess whether the benefits of meta-learning persist beyond the linearized regime.

4.4.1 Regulation under nonlinear dynamics

To assess the impact of model nonlinearities, a regulation task without external disturbances is first considered. As in Section 4.1, the neural network controller is trained using 27 agents whose initial positions are selected to best describe the cube $(-5, 5)^3$ m, including the vertices, face centers, edge centers, and the cube center (Figure 4.1). Two neural network controllers are trained independently. The first controller is trained using the linearized state-space model (2.19), while the second controller is trained using the full nonlinear dynamics (2.7)-(2.12).

Both controllers are then evaluated using 10 additional agents with randomly sampled initial positions within the same cube. In both cases, the same quadratic cost function (2.28) and identical weighting matrices Q and R are used. The resulting trajectories are shown in Figure 4.17.

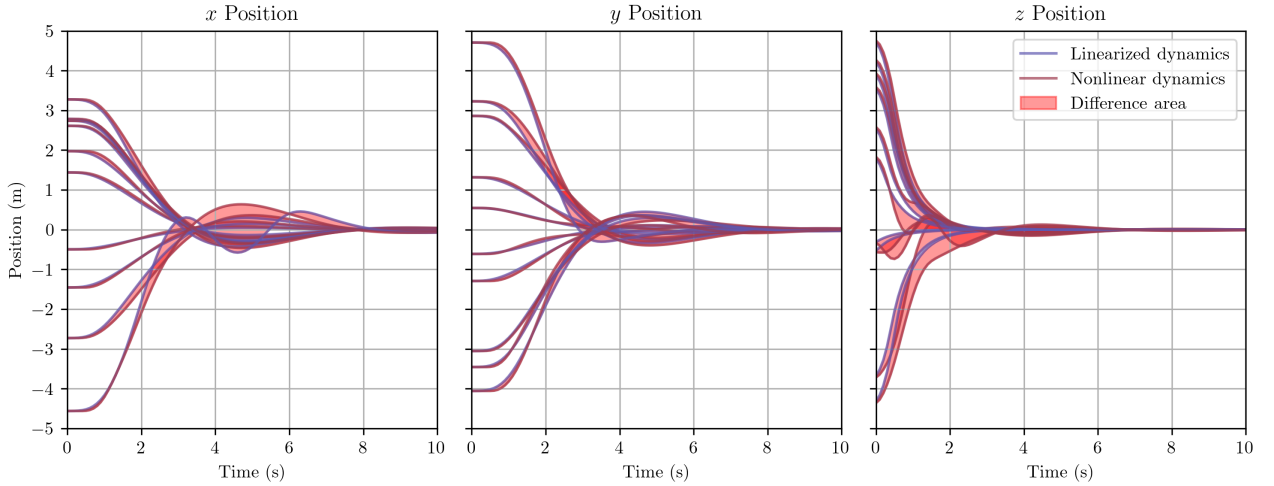


Figure 4.17: Regulation trajectories of 10 agents with random initial positions in the cube $(-5, 5)^3$ m, obtained using controllers trained on the linearized dynamics and on the full nonlinear dynamics.

The trajectories obtained with the linearized and nonlinear dynamics are very similar overall, indicating that the neural network controller successfully regulates the nonlinear system. Small differences can nevertheless be observed: trajectories generated using the nonlinear dynamics exhibit slightly more oscillatory behavior. This difference is expected, since the linearized model only approximates the dynamics locally around the equilibrium and does not capture all nonlinear coupling effects that can influence transient oscillations. This result is particularly encouraging, as it demonstrates that a neural network controller trained directly on nonlinear dynamics can achieve stable and accurate regulation. Such a property is highly relevant for real-world deployment, where the true system dynamics are inherently nonlinear and where reducing the sim-to-real gap is critical.

It should be noted, however, that training the neural network using the nonlinear dynamics requires approximately twice the computational time compared to training with the linearized model. This increase is likely due to the additional computational cost associated with evaluating trigonometric functions in the nonlinear equations of motion.

4.4.2 Wind disturbance: single-axis

This experiment reproduces the single-axis wind disturbance scenario introduced in Sub-section 4.3.1, using the same training and target task distributions (Figure 4.6), but replacing the linearized dynamics with the full nonlinear model (2.7)–(2.12). The objective is to assess whether the conclusions drawn in the linear setting remain valid when nonlinear dynamics are considered.

As in the linear case, both a jointly trained controller and a meta-trained controller are learned. Figures 4.18 and 4.19 report the joint-training and meta-training losses, respectively. When compared with their linear counterparts (Figures 4.7 and 4.8), the loss curves exhibit very similar profiles although the final converged loss value is slightly higher. This difference can be attributed to the influence of nonlinear effects on the transient response of the system, which tend to produce more oscillatory behavior. Overall, this indicates that the training behavior of both joint learning and meta-learning is largely unaffected by the introduction of nonlinear dynamics.

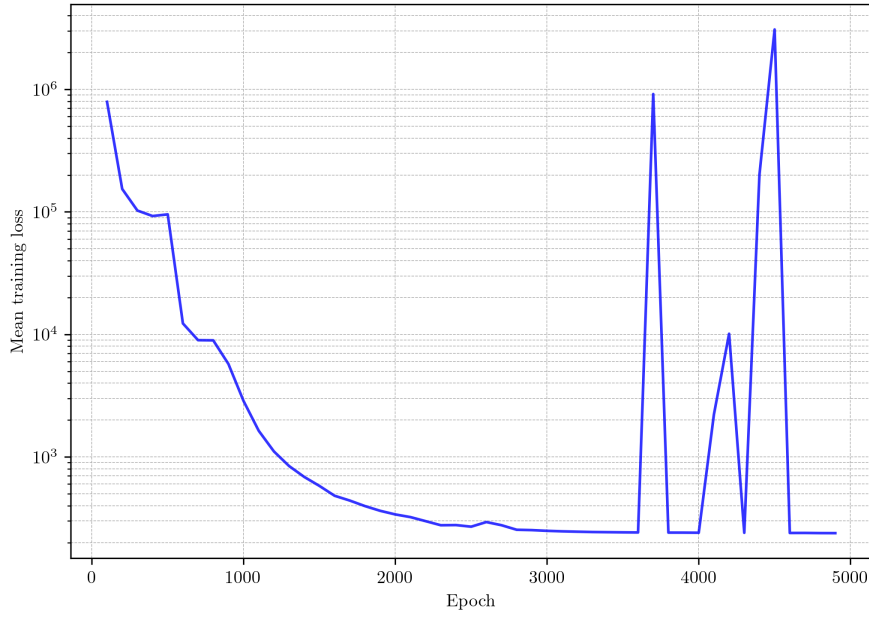


Figure 4.18: Nonlinear single-axis experiment: evolution of the joint training loss across all four training tasks.

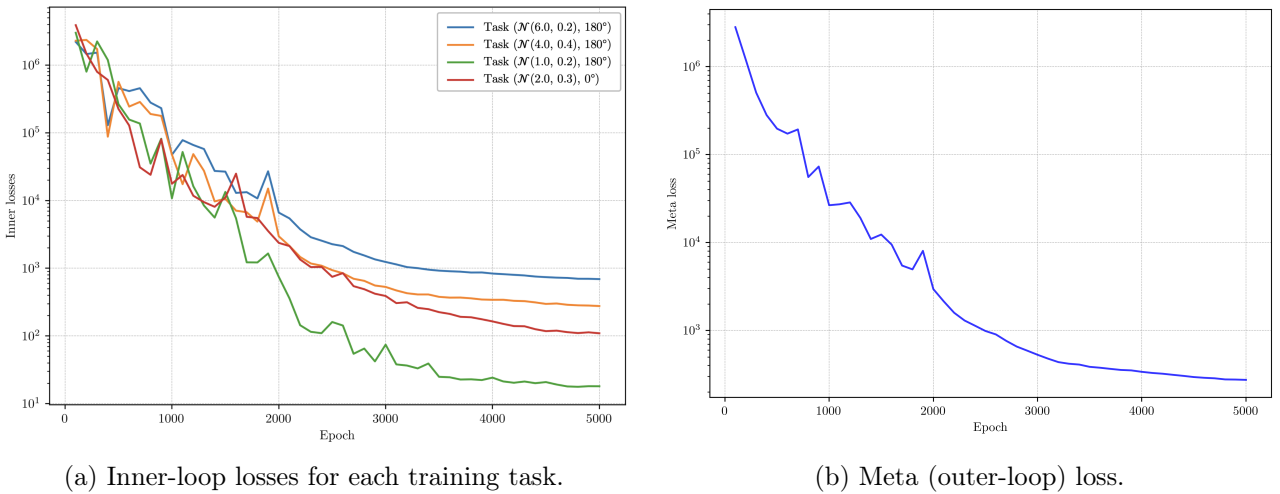


Figure 4.19: Nonlinear single-axis experiment: meta-training losses.

Once the joint-training and meta-training phases are completed, few-shot adaptation to the unseen target task $\mathcal{T}_{\text{target}}$ is performed. Starting from the parameters obtained after joint training and meta-training, respectively, five gradient update steps are applied. As in the linear case, each update uses five randomly sampled trajectories drawn from $\mathcal{T}_{\text{target}}$. After each adaptation step, the updated controller is evaluated on the full target task distribution. Figure 4.20 compares the resulting losses on $\mathcal{T}_{\text{target}}$ for the joint-trained and meta-trained initializations when using the nonlinear dynamics.

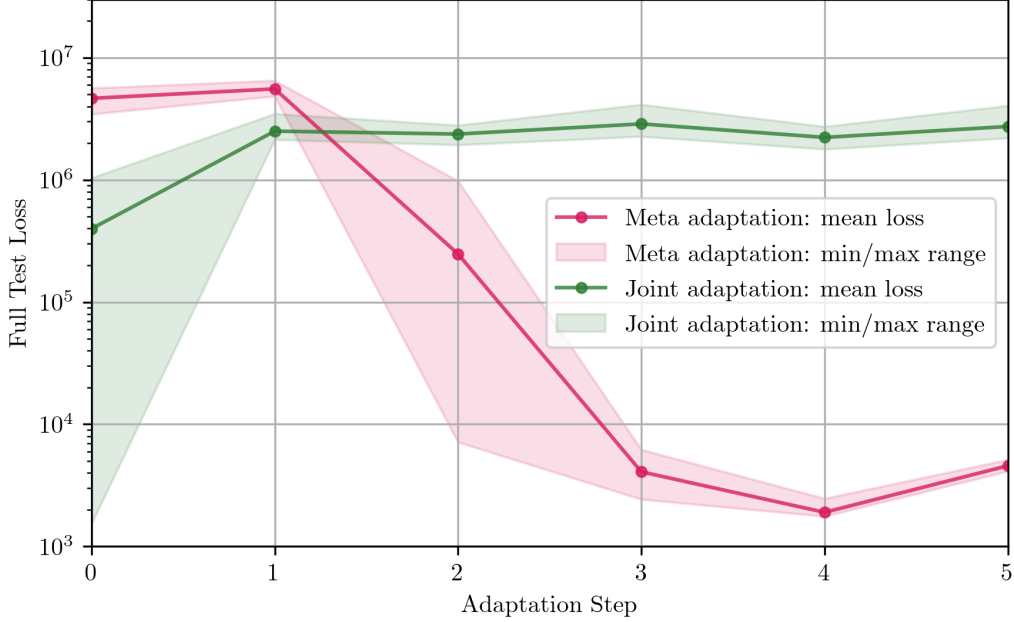


Figure 4.20: Nonlinear single-axis experiment: comparison of few-shot adaptation performance on the target task $\mathcal{T}_{\text{target}}$, starting from joint-trained and meta-trained initializations.

As in the linear case, the initial loss on $\mathcal{T}_{\text{target}}$ prior to any adaptation step is higher for the meta-trained initialization than for the jointly trained one. This behavior is expected, as the meta-trained parameters are optimized for rapid adaptation rather than immediate performance on unseen tasks. Figure 4.20 shows that the loss associated with the meta-trained initialization decreases very rapidly and reaches a plateau after three adaptation steps. The reduction in loss is more abrupt than in the linear experiment (Figure 4.9), although the initial loss prior to any adaptation step is much higher than in the linear case and the final converged loss value is also slightly higher. This difference is likely attributable to the more oscillatory transient behavior induced by the nonlinear dynamics. In contrast, the jointly trained initialization does not exhibit any significant decrease in loss during adaptation. Rather than oscillating as in the linear case, the loss remains at a consistently high level with very small variance. After five adaptation steps, the loss remains approximately three orders of magnitude larger than that obtained with the meta-trained initialization.

To further illustrate this difference in behavior, Figure 4.21 presents quadcopter trajectories subject to wind disturbances sampled from $\mathcal{T}_{\text{target}}$, using the parameters obtained after five adaptation steps. When starting from the meta-trained initialization, the controller successfully rejects the disturbance and maintains the agent close to the origin (Figure 4.21a). In contrast, the controller initialized from jointly trained parameters fails to compensate for the disturbance, resulting in diverging trajectories (Figure 4.21b). Moreover, when using the nonlinear dynamics, the resulting trajectories exhibit more pronounced oscillations than in the linear case (Figure 4.10). This behavior is consistent with the influence of nonlinear effects on the transient response of the system.

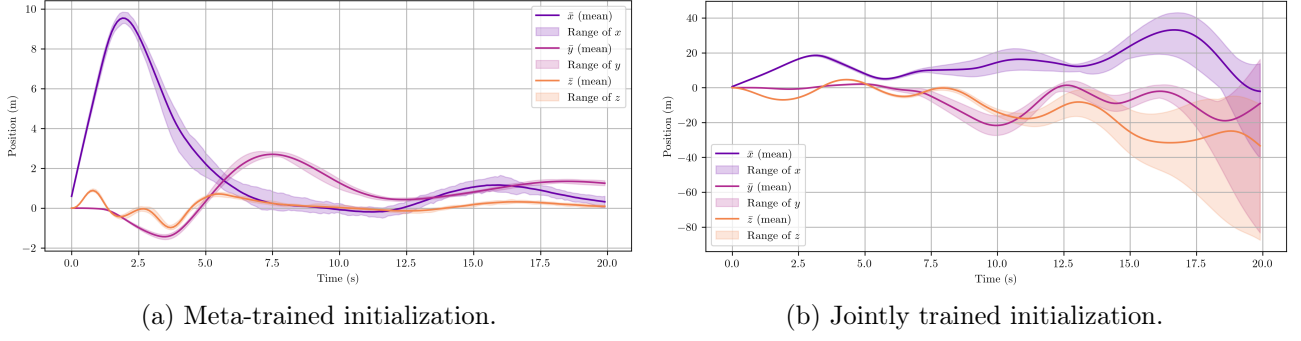


Figure 4.21: Nonlinear single-axis experiment: simulated quadcopter trajectories under wind disturbances sampled from $\mathcal{T}_{\text{target}}$, using parameters obtained after five adaptation steps.

Overall, these results demonstrate that the meta-learning framework remains fully effective when applied to nonlinear dynamics. The meta-trained initialization consistently enables rapid and reliable adaptation to unseen wind disturbances, closely mirroring the behavior observed in the linearized setting. The main drawback of using nonlinear dynamics lies in the increased computational cost: training with the nonlinear model requires approximately twice the computation time compared to the linearized case. Nevertheless, this cost is offset by the improved realism of the dynamics and the resulting reduction of the sim-to-real gap.

4.4.3 Wind disturbance: all directions

Finally, this last experiment reproduces the all-directions wind disturbance scenario introduced in subsection 4.3.2, using the same training and target task distributions (Figure 4.11), but replacing the linearized dynamics with the full nonlinear model (2.7)–(2.12). Once again, the objective is to evaluate whether the conclusions drawn in the linear all-directions setting remain valid when the full nonlinear dynamics are considered.

As in the linear all-directions experiment, both joint training and meta-training (reported in Figures 4.22 and 4.23) exhibit stable and well-behaved optimization dynamics when applied to the nonlinear model. The evolution of the joint-training loss and the meta-training losses closely matches the trends observed in the linear case (Figures 4.12 and 4.13), with both approaches converging to comparable loss levels on the training tasks. This indicates that the introduction of nonlinear dynamics does not significantly affect the training behavior of either method.

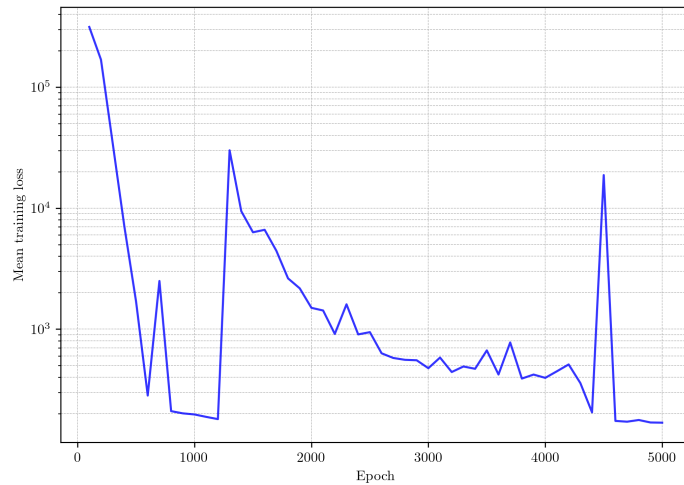


Figure 4.22: Nonlinear all-directions experiment: evolution of the joint training loss across all four training tasks.

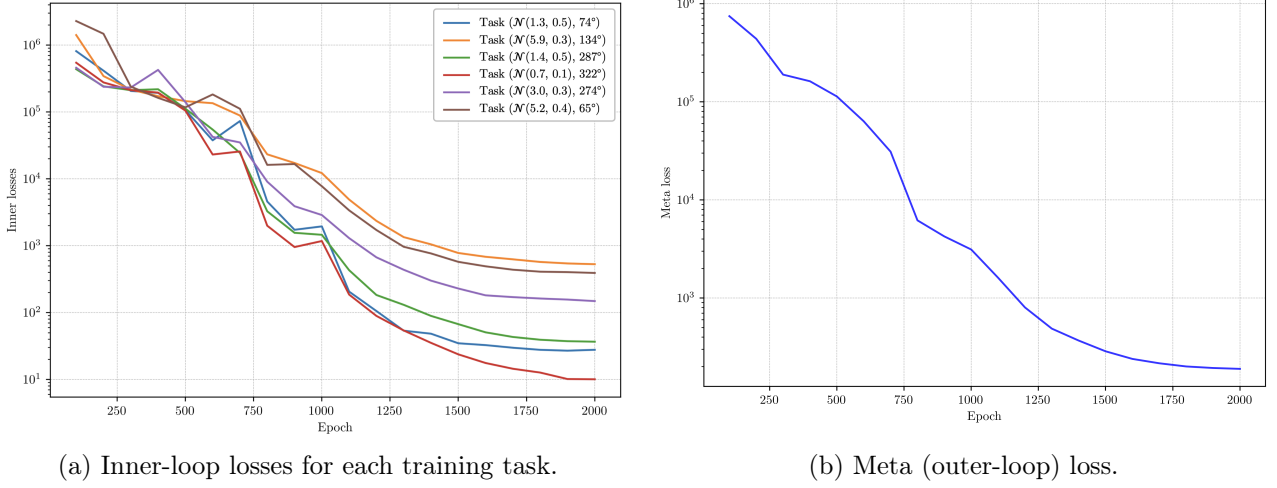


Figure 4.23: Nonlinear all-directions experiment: meta-training losses.

Following training, few-shot adaptation to the unseen target task $\mathcal{T}_{\text{target}}$ is performed using the same protocol as in the linear setting. Figure 4.24 shows that the qualitative behavior observed previously is preserved. Before adaptation, the meta-trained initialization yields a higher loss on the target task than the jointly trained initialization. However, after only a few adaptation steps, the loss associated with the meta-trained parameters decreases rapidly and consistently, reaching a low plateau. In contrast, the jointly trained initialization exhibits no improvement after the five adaptation steps. Although a slight decrease in loss can be observed, the resulting performance remains significantly worse than that obtained with the meta-trained initialization.

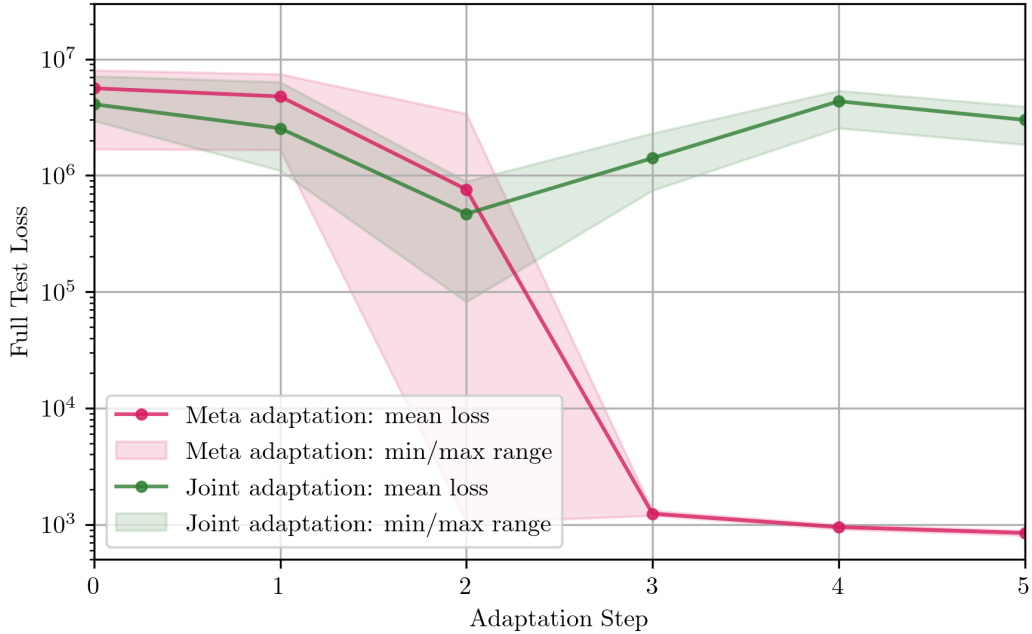


Figure 4.24: Nonlinear all-directions experiment: comparison of few-shot adaptation performance on the target task $\mathcal{T}_{\text{target}}$, starting from joint-trained and meta-trained initializations.

To further illustrate this difference in behavior, Figure 4.25 presents quadcopter trajectories subject to wind disturbances sampled from $\mathcal{T}_{\text{target}}$, using the parameters obtained after five adaptation steps. When starting from the meta-trained initialization, the controller successfully rejects the disturbance and maintains the agent close to the origin (Figure 4.25a) while the controller initialized from jointly trained parameters fails to compensate for the disturbance (Figure 4.25b).

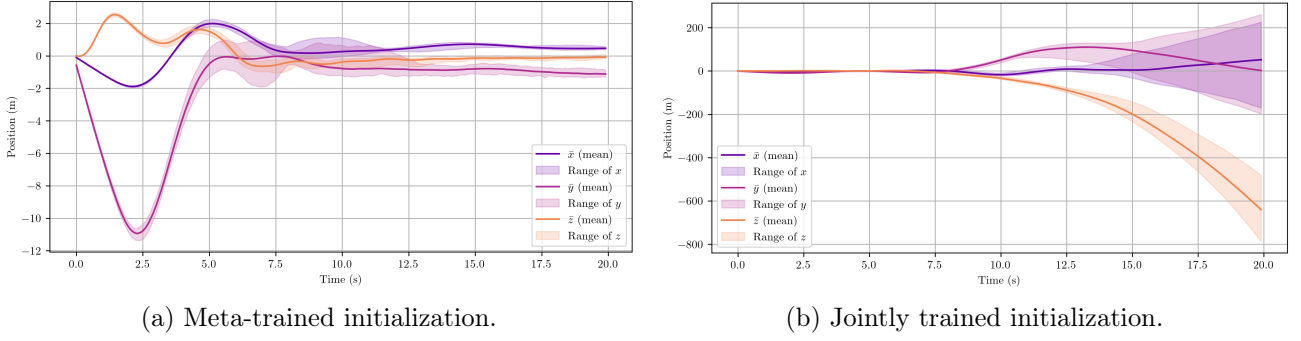


Figure 4.25: Nonlinear all-directions experiment: simulated quadcopter trajectories under wind disturbances sampled from $\mathcal{T}_{\text{target}}$, using parameters obtained after five adaptation steps.

Overall, these results demonstrate that the advantages of meta-learning observed in the linear all-directions experiment extend naturally to the nonlinear dynamics, confirming that MAML remains effective even in the presence of strong nonlinearities and unseen wind directions. However, more pronounced oscillations are once again observed compared to the linear case (Figure 4.15), likely due to the influence of nonlinear effects on the system’s transient response.

4.5 Additional observations and implementation notes

Several additional observations can be made regarding the meta-learning experiments and the training procedure.

Across all joint training experiments, noticeable loss spikes are observed in the training curves (Figures 4.7, 4.12, 4.18, and 4.22). Although the overall loss decreases as expected, occasional large spikes appear throughout training. This behavior can be explained by the task heterogeneity inherent to joint training. During optimization, training samples are randomly drawn from multiple task distributions. When several consecutive updates are performed using samples from similar tasks, the controller parameters become specialized to that region of the task space. If a subsequent update is then computed using samples from a significantly different task, the current parameters may be poorly suited to that task, resulting in a sudden increase in the loss.

In all MAML experiments, the input cost matrix R in the quadratic cost function is set to zero. This choice reflects the objective of the simulations, which is solely to maintain the quadcopter at the origin under wind disturbances, without penalizing control effort. Since the control inputs are explicitly constrained, the absence of an input penalty does not lead to instability or unbounded control actions. In worst-case scenarios, the controller saturates the inputs at their maximum allowable values, but the system remains numerically stable throughout the simulations.

The performance of the meta-learning algorithm is sensitive to several key hyperparameters. In particular, the number of meta-training epochs must be sufficiently large for the loss to reach a clear plateau, but excessively large values only increase computation time without improving performance. The inner-loop and outer-loop learning rates, denoted by α and β , respectively, are chosen in the range $[10^{-3}, 2 \times 10^{-3}]$. Learning rates that are too large lead to oscillatory behavior and prevent convergence, while overly small values result in slow progress and insufficient optimization within the number of epochs. The number of samples used during meta-training is fixed to $K_{\text{inner}} = 5$ for the inner loop and $K_{\text{outer}} = 5$ for the outer loop. Given that each task contains 50 trajectories, this choice provides a reasonable trade-off between stochasticity and computational efficiency, while remaining consistent with the few-shot learning setting. During the adaptation phase, the number of adaptation samples is set to $K_{\text{inner}} = 5$. This choice is deliberate and emphasizes the minimal-data regime targeted in this work. The inner-loop adaptation learning rate was tuned depending on the experiment, with values selected in the range $[5 \times 10^{-3}, 8 \times 10^{-2}]$.

5. Conclusion

This project investigated the use of meta-learning for wind disturbance rejection in quadcopter control, with the objective of enabling rapid adaptation to previously unseen environments using only a small amount of data. To this end, a complete simulation framework was developed from scratch, including the implementation of a quadcopter model from the literature, neural network-based controllers, and a meta-learning pipeline based on the Model-Agnostic Meta-Learning (MAML) algorithm.

The work began with the implementation of a linearized quadcopter model and a multilayer perceptron (MLP) controller, which was first validated on a simple regulation task through comparison with a classical Linear Quadratic Regulator (LQR). This comparison showed that the neural network controller achieves regulation performance comparable to that of the LQR, thereby validating the learning-based control framework. A series of motivation experiments then highlighted the limitations of standard joint training in scenarios with highly unbalanced data availability across disturbance conditions. In particular, joint training failed to generalize when only a few samples were available for certain wind directions, motivating the use of meta-learning. The Model-Agnostic Meta-Learning (MAML) algorithm was subsequently applied to wind disturbance rejection problems of increasing difficulty. In both single-axis and all-directions experiments, meta-learning enabled rapid adaptation to unseen tasks using as few as five samples and five gradient steps, while jointly trained controllers failed to do so. Robustness analyses further showed that the adaptation performance was largely insensitive to the specific choice of adaptation samples. The project further investigated the impact of model fidelity by comparing results obtained with linearized dynamics to those obtained with the full nonlinear quadcopter model. The regulation experiments revealed that trajectories produced under linearized and nonlinear dynamics are very similar, confirming that the linearization constitutes a good approximation around the operating point. Repeating the meta-learning experiments with the nonlinear dynamics showed that MAML retains its advantages, with rapid and reliable adaptation observed in both the single-axis and all-directions scenarios. The main trade-off observed is computational, as simulations involving the nonlinear dynamics require approximately twice the computation time.

Several directions for future work naturally emerge from this study. First, more detailed and realistic nonlinear models could be considered in order to better capture real quadcopter dynamics and actuator limitations. Second, incorporating formal stability guarantees for the neural network controller would be essential for real-world deployment, ensuring that the controlled system remains stable under all admissible operating conditions. Finally, deploying the proposed approach on real hardware would allow for a direct evaluation of the sim-to-real gap and would constitute a crucial step toward practical, real-world applications of meta-learning-based control.

References

- [1] Faraz Ahmad, Pushpendra Kumar, Anamika Bhandari, and Pravin P. Patil. Simulation of the quadcopter dynamics with lqr based control. *Materials Today: Proceedings*, 24:326–332, 2020.
- [2] Mina Kamel, Michael Burri, and Roland Siegwart. Linear vs nonlinear mpc for trajectory tracking applied to rotary wing micro aerial vehicles. *arXiv preprint arXiv:1611.09240*, 2017.
- [3] Philipp Foehn and Davide Scaramuzza. Onboard state dependent lqr for agile quadrotors. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 6566–6572. IEEE, 2018.
- [4] Michael O’Connell, Guanya Shi, Xichen Shi, and Soon-Jo Chung. Meta-learning-based robust adaptive flight control under uncertain wind conditions. *arXiv preprint arXiv:2103.01932*, 2021.
- [5] Michael O’Connell, Guanya Shi, Xichen Shi, Kamyar Azizzadenesheli, Anima Anandkumar, Yisong Yue, and Soon-Jo Chung. Neural-fly enables rapid learning for agile flight in strong winds. *arXiv preprint arXiv:2205.06908*, 2022.
- [6] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1126–1135. PMLR, 2017.
- [7] Alex Nichol, Joshua Achiam, and John Schulman. On first-order meta-learning algorithms. *arXiv preprint arXiv:1803.02999*, 2018.
- [8] Chelsea Finn, Kelvin Xu, and Sergey Levine. Probabilistic model-agnostic meta-learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

Appendix

5.1 LQR formulation

For the discrete-time linear system

$$\mathbf{x}_{k+1} = A_d \mathbf{x}_k + B_d \mathbf{u}_k,$$

the Linear Quadratic Regulator (LQR) minimizes the infinite-horizon quadratic cost

$$J = \sum_{k=0}^{\infty} \left(\mathbf{x}_k^\top Q \mathbf{x}_k + \mathbf{u}_k^\top R \mathbf{u}_k \right).$$

The optimal state-feedback gain is obtained by solving the discrete algebraic Riccati equation

$$P = A_d^\top P A_d - A_d^\top P B_d (R + B_d^\top P B_d)^{-1} B_d^\top P A_d + Q,$$

leading to the feedback gain

$$K = (R + B_d^\top P B_d)^{-1} B_d^\top P A_d.$$

The weighting matrices Q and R are chosen using inverse squared normalization of the state and input limits:

$$Q_{ii} = \frac{1}{(x_{i,\max})^2}, \quad R_{jj} = \frac{1}{(u_{j,\max})^2}.$$

5.2 Gantt chart

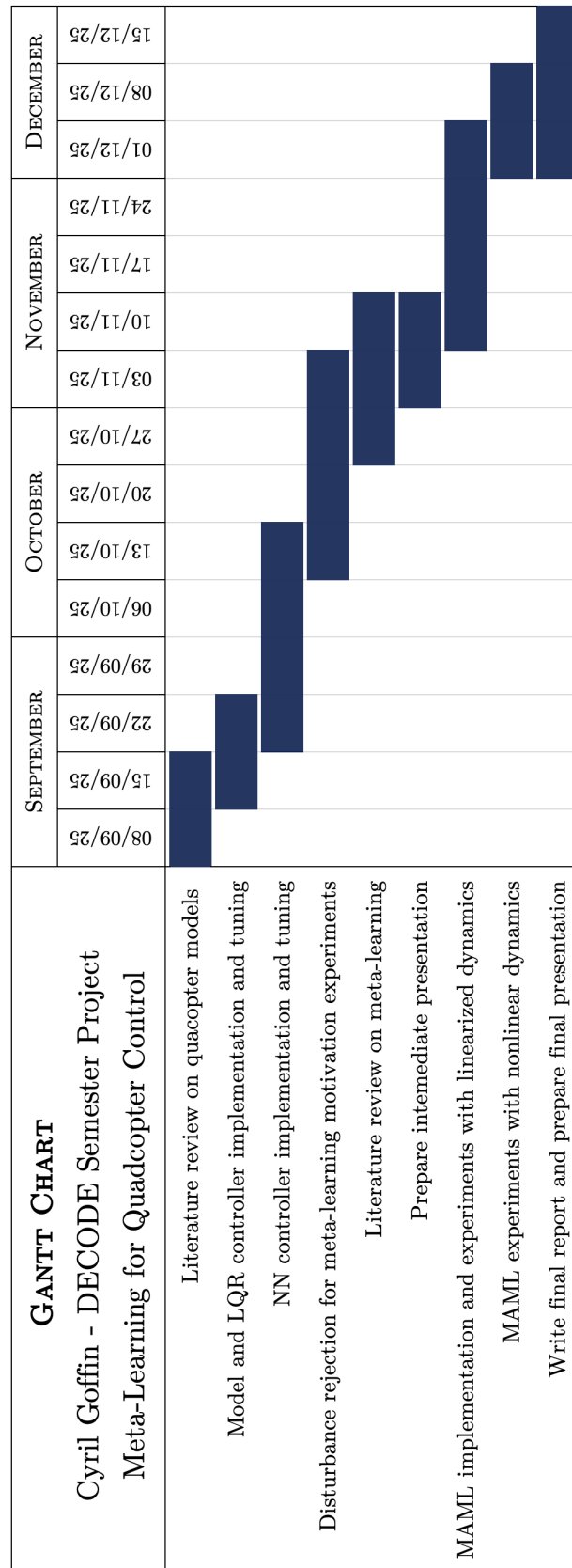


Figure 5.1: Gantt chart of the project.